Kanga_Kouame-Ghyslain_1_notebook_112025

November 29, 2025

1 OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'un instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)…

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

[17]:
```
#Importation de la librairie Pandas
import pandas as pd
```

[18]:
```
#Importation des librairies pour la suite finale du projet
import geopandas as gpd
import geodatasets
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

1.2 - Chargement des fichiers Excel

[19]:
```
#Importation du fichier population.csv
```

```
population = pd.read_csv('D:/dataAnalyst/Projet4/travail/DAN-P4-FAO/population.
 ↪csv')

#Importation du fichier dispo_alimentaire.csv
dispo_alimentaire = pd.read_csv('D:/dataAnalyst/Projet4/travail/DAN-P4-FAO/
 ↪dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
aide_alimentaire = pd.read_csv('D:/dataAnalyst/Projet4/travail/DAN-P4-FAO/
 ↪aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous_nutrition = pd.read_csv('D:/dataAnalyst/Projet4/travail/DAN-P4-FAO/
 ↪sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

[20]:
```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(population.
 ↪shape[0]))
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

```
Le tableau comporte 1416 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)
```

[21]:
```
#Consulter le nombre de colonnes
print("Le tableau comporte {} colonnes".format(population.shape[1]))
#La nature des données dans chacune des colonnes
population.dtypes
##ou population.info() pour avoir plus d'informations sur le tableau
#Le nombre de valeurs présentes dans chacune des colonnes
population.count()
```

```
Le tableau comporte 3 colonnes
```

[21]:
```
Zone      1416
Année     1416
Valeur    1416
dtype: int64
```

[22]:
```
#Affichage les 5 premières lignes de la table
population.head()
```

[22]:

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603

```
3  Afghanistan   2016  35383.032
4  Afghanistan   2017  36296.113
```

[23]:
```
#Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier␣
↪la population par 1000
#Multiplication de la colonne valeur par 1000
population['Valeur'] = population['Valeur'] * 1000
print(population.head())
```

```
          Zone  Année      Valeur
0  Afghanistan   2013  32269589.0
1  Afghanistan   2014  33370794.0
2  Afghanistan   2015  34413603.0
3  Afghanistan   2016  35383032.0
4  Afghanistan   2017  36296113.0
```

[24]:
```
#changement du nom de la colonne Valeur par Population
# population.rename(columns={"Valeur": "Population"}, inplace=True) Méthode ne␣
↪gardant pas en mémoire
#dans le fichier initial si changement de DataFrame
population = population.rename(columns={"Valeur": "Population"})
```

[25]:
```
#Affichage les 5 premières lignes de la table pour voir les modifications
print(population.head())
```

```
          Zone  Année  Population
0  Afghanistan   2013  32269589.0
1  Afghanistan   2014  33370794.0
2  Afghanistan   2015  34413603.0
3  Afghanistan   2016  35383032.0
4  Afghanistan   2017  36296113.0
```

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

[26]:
```
#Afficher les dimensions du dataset
#rows = lignes, cols = colonnes
rows, cols = dispo_alimentaire.shape
print(f"Le dataset (fichier disponibilité alimentaire) contient {rows} lignes␣
↪et {cols} colonnes.")
```

```
Le dataset (fichier disponibilité alimentaire) contient 15605 lignes et 18
colonnes.
```

[27]:
```
#Consulter le nombre de colonnes
print("Le nombre de colonnes est : {}".format(dispo_alimentaire.shape[1]))
```

```
Le nombre de colonnes est : 18
```

[28]:
```
#Afficher les 5 premières lignes de la table
dispo_alimentaire.head()
```

[28]:

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale	NaN	
1	Afghanistan	Agrumes, Autres	vegetale	NaN	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	
3	Afghanistan	Ananas	vegetale	NaN	
4	Afghanistan	Bananes	vegetale	NaN	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	NaN	5.0	
1	NaN	1.0	
2	NaN	1.0	
3	NaN	0.0	
4	NaN	4.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
0	0.20	
1	0.01	
2	0.01	
3	NaN	
4	0.02	

	Disponibilité de protéines en quantité (g/personne/jour)	\
0	0.77	
1	0.02	
2	0.03	
3	NaN	
4	0.05	

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	\
0	53.0	NaN	NaN	
1	41.0	2.0	40.0	
2	2.0	NaN	2.0	
3	0.0	NaN	0.0	
4	82.0	NaN	82.0	

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53.0	NaN	53.0	NaN	NaN	NaN
1	39.0	2.0	3.0	NaN	NaN	NaN
2	2.0	NaN	NaN	NaN	NaN	NaN
3	0.0	NaN	NaN	NaN	NaN	NaN

```
4        82.0     NaN         NaN       NaN         NaN                  NaN
```

[31]:
```python
#remplacement des NaN dans le dataset par des 0
dispo_alimentaire = dispo_alimentaire.fillna(0)
dispo_alimentaire.head()
```

[31]:
```
          Zone                Produit   Origine  Aliments pour animaux  \
0  Afghanistan       Abats Comestible   animale                    0.0
1  Afghanistan        Agrumes, Autres  vegetale                    0.0
2  Afghanistan  Aliments pour enfants  vegetale                    0.0
3  Afghanistan                 Ananas  vegetale                    0.0
4  Afghanistan                Bananes  vegetale                    0.0

   Autres Utilisations  Disponibilité alimentaire (Kcal/personne/jour)  \
0                  0.0                                             5.0
1                  0.0                                             1.0
2                  0.0                                             1.0
3                  0.0                                             0.0
4                  0.0                                             4.0

   Disponibilité alimentaire en quantité (kg/personne/an)  \
0                                                   1.72
1                                                   1.29
2                                                   0.06
3                                                   0.00
4                                                   2.70

   Disponibilité de matière grasse en quantité (g/personne/jour)  \
0                                                   0.20
1                                                   0.01
2                                                   0.01
3                                                   0.00
4                                                   0.02

   Disponibilité de protéines en quantité (g/personne/jour)  \
0                                                   0.77
1                                                   0.02
2                                                   0.03
3                                                   0.00
4                                                   0.05

   Disponibilité intérieure  Exportations - Quantité  Importations - Quantité  \
0                      53.0                       0.0                       0.0
1                      41.0                       2.0                      40.0
2                       2.0                       0.0                       2.0
3                       0.0                       0.0                       0.0
4                      82.0                       0.0                      82.0
```

```
   Nourriture  Pertes  Production  Semences  Traitement  Variation de stock
0        53.0     0.0        53.0       0.0         0.0                 0.0
1        39.0     2.0         3.0       0.0         0.0                 0.0
2         2.0     0.0         0.0       0.0         0.0                 0.0
3         0.0     0.0         0.0       0.0         0.0                 0.0
4        82.0     0.0         0.0       0.0         0.0                 0.0
```

[32]:
```python
#multiplication de toutes les lignes contenant des milliers de tonnes en Kg
#1T= 1000kg; 1000T=1 000 000kg
### Je commence par créer une variable col_tonnes pour stocker les colonnes en␣
↪tonnes
col_tonnes = [
    'Aliments pour animaux',
    'Disponibilité intérieure',
    'Exportations - Quantité',
    'Importations - Quantité',
    'Nourriture',
    'Pertes',
    'Production',
    'Semences',
    'Traitement',
    'Variation de stock'
]
#On sait que 1 tonne = 1000 kg => 1000 tonnes = 1000 000 Kg
#On va donc faire une boucle sur la variable précédente pour répondre à la␣
↪question
#dispo_alimentaire['Production (tonnes)'] = dispo_alimentaire['Production␣
↪(tonnes)'] * 1000 "Ceci ne fonctionne pas
for col in col_tonnes:
    if col in dispo_alimentaire.columns: #Vérif
        dispo_alimentaire[col] = dispo_alimentaire[col] * 1000000
dispo_alimentaire[col_tonnes].head()
```

[32]:
```
   Aliments pour animaux  Disponibilité intérieure  Exportations - Quantité  \
0                    0.0                53000000.0                      0.0
1                    0.0                41000000.0                2000000.0
2                    0.0                 2000000.0                      0.0
3                    0.0                       0.0                      0.0
4                    0.0                82000000.0                      0.0

   Importations - Quantité  Nourriture     Pertes  Production  Semences  \
0                      0.0  53000000.0        0.0  53000000.0       0.0
1               40000000.0  39000000.0  2000000.0   3000000.0       0.0
2                2000000.0   2000000.0        0.0         0.0       0.0
3                      0.0         0.0        0.0         0.0       0.0
4               82000000.0  82000000.0        0.0         0.0       0.0
```

```
   Traitement  Variation de stock
0         0.0                 0.0
1         0.0                 0.0
2         0.0                 0.0
3         0.0                 0.0
4         0.0                 0.0
```

[33]:
```python
#Affichage des 5 premières lignes de la table
dispo_alimentaire.head()
```

[33]:
```
          Zone                Produit   Origine  Aliments pour animaux  \
0  Afghanistan       Abats Comestible   animale                    0.0
1  Afghanistan        Agrumes, Autres  vegetale                    0.0
2  Afghanistan  Aliments pour enfants  vegetale                    0.0
3  Afghanistan                 Ananas  vegetale                    0.0
4  Afghanistan                Bananes  vegetale                    0.0

   Autres Utilisations  Disponibilité alimentaire (Kcal/personne/jour)  \
0                  0.0                                             5.0
1                  0.0                                             1.0
2                  0.0                                             1.0
3                  0.0                                             0.0
4                  0.0                                             4.0

   Disponibilité alimentaire en quantité (kg/personne/an)  \
0                                               1.72
1                                               1.29
2                                               0.06
3                                               0.00
4                                               2.70

   Disponibilité de matière grasse en quantité (g/personne/jour)  \
0                                               0.20
1                                               0.01
2                                               0.01
3                                               0.00
4                                               0.02

   Disponibilité de protéines en quantité (g/personne/jour)  \
0                                               0.77
1                                               0.02
2                                               0.03
3                                               0.00
4                                               0.05

   Disponibilité intérieure  Exportations - Quantité  Importations - Quantité  \
```

```
0                53000000.0                     0.0                     0.0
1                41000000.0               2000000.0              40000000.0
2                 2000000.0                     0.0               2000000.0
3                       0.0                     0.0                     0.0
4                82000000.0                     0.0              82000000.0

   Nourriture     Pertes  Production  Semences  Traitement  Variation de stock
0  53000000.0        0.0  53000000.0       0.0         0.0                 0.0
1  39000000.0  2000000.0   3000000.0       0.0         0.0                 0.0
2   2000000.0        0.0         0.0       0.0         0.0                 0.0
3         0.0        0.0         0.0       0.0         0.0                 0.0
4  82000000.0        0.0         0.0       0.0         0.0                 0.0
```

2.3 - Analyse exploratoire du fichier aide alimentaire

[34]:
```python
#Afficher les dimensions du dataset
rows, cols = aide_alimentaire.shape
print(f"Le dataset (fichier aide alimentaire) contient {rows} lignes et {cols}␣
↪colonnes.")
```

```
Le dataset (fichier aide alimentaire) contient 1475 lignes et 4 colonnes.
```

[35]:
```python
#Consulter le nombre de colonnes
print("Le nombre de colonnes est : {}".format(aide_alimentaire.shape[1]))
```

```
Le nombre de colonnes est : 4
```

[36]:
```python
#Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

[36]:

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

[37]:
```python
#changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={'Pays bénéficiaire': 'Zone'}, inplace=True)
aide_alimentaire.head()
```

[37]:

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

[38]:
```
#Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000␣
↪pour avoir des kg
aide_alimentaire["Valeur_kg"] = aide_alimentaire["Valeur"] * 1000
```

[39]:
```
#Affichage des 5 premières lignes de la table
aide_alimentaire.head()
```

[39]:

	Zone	Année	Produit	Valeur	Valeur_kg
0	Afghanistan	2013	Autres non-céréales	682	682000
1	Afghanistan	2014	Autres non-céréales	335	335000
2	Afghanistan	2013	Blé et Farin	39224	39224000
3	Afghanistan	2014	Blé et Farin	15160	15160000
4	Afghanistan	2013	Céréales	40504	40504000

2.3 - Analyse exploratoire du fichier sous nutrition

[40]:
```
#Afficher les dimensions du dataset
rows, cols = sous_nutrition.shape
print(f"Le dataset (fichier sous_nutrition) contient {rows} lignes et {cols}␣
↪colonnes.")
```

```
Le dataset (fichier sous_nutrition) contient 1218 lignes et 3 colonnes.
```

[41]:
```
#Consulter le nombre de colonnes
print("Le nombre de colonnes est : {}".format(sous_nutrition.shape[1]))
```

```
Le nombre de colonnes est : 3
```

[42]:
```
#Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

[42]:

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

[43]:
```
#Conversion de la colonne sous nutrition en numérique
#sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur']) # Ce script␣
↪me retourne une erreur
# Nettoyer la colonne Valeur (supprimer espaces)
sous_nutrition["Valeur"] = sous_nutrition["Valeur"].astype(str).str.strip()

# Remplacer les cas particuliers "<0.1" par une valeur numérique
# Ici je mets 0.1, mais tu peux choisir 0.05 si tu préfères
sous_nutrition["Valeur"] = sous_nutrition["Valeur"].replace({"<0.1": "0.1"})
# Vérification rapide
print(sous_nutrition.head())
```

```
          Zone      Année Valeur
0  Afghanistan  2012-2014    8.6
1  Afghanistan  2013-2015    8.8
2  Afghanistan  2014-2016    8.9
3  Afghanistan  2015-2017    9.7
4  Afghanistan  2016-2018   10.5
```

[44]:
```
#Conversion de la colonne (avec l'argument errors=coerce qui permet de␣
↪convertir automatiquement les lignes qui ne sont pas des nombres en NaN)
#Puis remplacement des NaN en 0

# Conversion en numérique, les erreurs deviennent NaN
sous_nutrition["Valeur"] = pd.to_numeric(sous_nutrition["Valeur"],␣
↪errors="coerce")
#On pourrait vérifier la conversion des données.
print(sous_nutrition["Valeur"].dtype) #si float64, alors la conversion a bien␣
↪fonctionnée
print(sous_nutrition["Valeur"].describe()) # Pour les statistiques
print(sous_nutrition[sous_nutrition["Valeur"].isna()].head())  # Si des valeurs␣
↪non converties

# Remplacer les NaN (cellules vides) par 0
sous_nutrition["Valeur"] = sous_nutrition["Valeur"].fillna(0)
print(sous_nutrition.head())
```

```
float64
count    624.000000
mean       5.123077
std       19.445818
min        0.100000
25%        0.275000
50%        1.100000
75%        3.900000
max      203.800000
Name: Valeur, dtype: float64
         Zone      Année  Valeur
24  Allemagne  2012-2014     NaN
25  Allemagne  2013-2015     NaN
26  Allemagne  2014-2016     NaN
27  Allemagne  2015-2017     NaN
28  Allemagne  2016-2018     NaN
          Zone      Année  Valeur
0  Afghanistan  2012-2014     8.6
1  Afghanistan  2013-2015     8.8
2  Afghanistan  2014-2016     8.9
3  Afghanistan  2015-2017     9.7
4  Afghanistan  2016-2018    10.5
```

[45]:
```
#changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition.rename(columns={'Valeur': 'sous_nutrition'}, inplace=True)
#Vérification de la commande
print(sous_nutrition.columns)
```

```
Index(['Zone', 'Année', 'sous_nutrition'], dtype='object')
```

[46]:
```
#Multiplication de la colonne sous_nutrition par 1000000
# Multiplication de la colonne sous_nutrition par 1 000 000
sous_nutrition["sous_nutrition"] = sous_nutrition["sous_nutrition"] * 1000000
```

[47]:
```
#Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

[47]:

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition

[48]:
```
# Il faut tout d'abord faire une jointure entre la table population et la table␣
↪sous-nutrition,
# en ciblant l'année 2017

# Filtrer sur l'année 2017 (population) et sur la période 2017-2019␣
↪(sous-nutrition)
pop_2017 = population[population["Année"] == 2017]
sn_2017 = sous_nutrition[sous_nutrition["Année"] == "2017-2019"]

# Jointure sur la colonne Zone (pays)
df_2017 = pd.merge(sn_2017, pop_2017, on="Zone", how="inner")
```

[49]:
```
#Affichage du dataset
print(df_2017)
```

```
                                        Zone    Année_x  sous_nutrition  \
0                                Afghanistan  2017-2019      11100000.0
1                             Afrique du Sud  2017-2019       3300000.0
2                                    Albanie  2017-2019        100000.0
3                                    Algérie  2017-2019       1200000.0
4                                  Allemagne  2017-2019             0.0
..                                        …          …               …
198  Venezuela (République bolivarienne du)  2017-2019       9100000.0
199                                 Viet Nam  2017-2019       6100000.0
200                                    Yémen  2017-2019             0.0
201                                   Zambie  2017-2019             0.0
```

```
202                                Zimbabwe  2017-2019             0.0

     Année_y  Population
0       2017  36296113.0
1       2017  57009756.0
2       2017   2884169.0
3       2017  41389189.0
4       2017  82658409.0
..       …         …
198     2017  29402484.0
199     2017  94600648.0
200     2017  27834819.0
201     2017  16853599.0
202     2017  14236595.0

[203 rows x 5 columns]
```

[50]:
```
#Calcul et affichage du nombre de personnes en état de sous nutrition
total_sous_nutrition = df_2017["sous_nutrition"].sum()
print(total_sous_nutrition)
```

```
546100000.0
```

[55]:
```
### Calcul de la proportion de personnes en sous-nutrition
# Calcul de la population mondiale totale en 2017
pop_mondiale_2017 = pop_2017["Population"].sum()

#Je Convertis pop_2017 avant de calculer la proportion
pop_mondiale_2017 = float(pop_mondiale_2017)
proportion_sous_nutrition = (total_sous_nutrition / pop_mondiale_2017) * 100

# Affichage du résultat
print("Population mondiale en 2017 :", pop_mondiale_2017)
print("Nombre de personnes en sous-nutrition en 2017 :", total_sous_nutrition)
print("Proportion mondiale en sous-nutrition en 2017 : {:.2f}%".
 ↪format(proportion_sous_nutrition))
```

```
Population mondiale en 2017 : 7548134111.0
Nombre de personnes en sous-nutrition en 2017 : 546100000.0
Proportion mondiale en sous-nutrition en 2017 : 7.23%
```

3.2 - Nombre théorique de personne qui pourrait être nourries

[57]:
```
#Combien mange en moyenne un être humain ? Source =>

#En moyenne, un être humain adulte consomme entre 2 000 et 2 700 kilocalories␣
 ↪par jour,
#selon le sexe et le niveau d'activité.
#Les femmes ont généralement besoin de 1 800 à 2 200 kcal/jour,
```

```
#tandis que les hommes nécessitent environ 2 200 à 2 700 kcal/jour
```

[58]:
```
dispo_alimentaire.head()
```

[58]:
```
          Zone                 Produit   Origine  Aliments pour animaux  \
0  Afghanistan       Abats Comestible   animale                    0.0
1  Afghanistan        Agrumes, Autres  vegetale                    0.0
2  Afghanistan  Aliments pour enfants  vegetale                    0.0
3  Afghanistan                 Ananas  vegetale                    0.0
4  Afghanistan                Bananes  vegetale                    0.0

   Autres Utilisations  Disponibilité alimentaire (Kcal/personne/jour)  \
0                  0.0                                             5.0
1                  0.0                                             1.0
2                  0.0                                             1.0
3                  0.0                                             0.0
4                  0.0                                             4.0

   Disponibilité alimentaire en quantité (kg/personne/an)  \
0                                                   1.72
1                                                   1.29
2                                                   0.06
3                                                   0.00
4                                                   2.70

   Disponibilité de matière grasse en quantité (g/personne/jour)  \
0                                                   0.20
1                                                   0.01
2                                                   0.01
3                                                   0.00
4                                                   0.02

   Disponibilité de protéines en quantité (g/personne/jour)  \
0                                                   0.77
1                                                   0.02
2                                                   0.03
3                                                   0.00
4                                                   0.05

   Disponibilité intérieure  Exportations - Quantité  Importations - Quantité  \
0               53000000.0                       0.0                       0.0
1               41000000.0                 2000000.0                40000000.0
2                2000000.0                       0.0                 2000000.0
3                      0.0                       0.0                       0.0
4               82000000.0                       0.0                82000000.0

   Nourriture     Pertes  Production  Semences  Traitement  Variation de stock
```

```
0  53000000.0        0.0  53000000.0       0.0         0.0                 0.0
1  39000000.0  2000000.0   3000000.0       0.0         0.0                 0.0
2   2000000.0        0.0         0.0       0.0         0.0                 0.0
3         0.0        0.0         0.0       0.0         0.0                 0.0
4  82000000.0        0.0         0.0       0.0         0.0                 0.0
```

[59]:
```
#On commence par faire une jointure entre le data frame population et␣
↪Dispo_alimentaire afin d'ajouter dans ce dernier la population

# Filtrer la population sur l'année 2017
pop_2017 = population[population["Année"] == 2017]

# Jointure entre dispo_alimentaire et population
dispo_pop = pd.merge(
    dispo_alimentaire,
    pop_2017,
    on="Zone",    # clé commune
    how="left"    # garde toutes les lignes de dispo_alimentaire
)
# Je joins les données de population 2017 au fichier des disponibilités␣
↪alimentaires,
#afin de pouvoir calculer les disponibilités par habitant. Cette étape est␣
↪essentielle pour comparer
#les volumes disponibles avec les besoins nutritionnels moyens.
```

[60]:
```
#Affichage du nouveau dataframe
print(dispo_pop.head())
print(dispo_pop.info())
```

```
          Zone                 Produit   Origine  Aliments pour animaux  \
0  Afghanistan       Abats Comestible   animale                    0.0
1  Afghanistan        Agrumes, Autres  vegetale                    0.0
2  Afghanistan  Aliments pour enfants  vegetale                    0.0
3  Afghanistan                 Ananas  vegetale                    0.0
4  Afghanistan                Bananes  vegetale                    0.0

   Autres Utilisations  Disponibilité alimentaire (Kcal/personne/jour)  \
0                  0.0                                             5.0
1                  0.0                                             1.0
2                  0.0                                             1.0
3                  0.0                                             0.0
4                  0.0                                             4.0

   Disponibilité alimentaire en quantité (kg/personne/an)  \
0                                               1.72
1                                               1.29
2                                               0.06
3                                               0.00
```

```
4                                                 2.70

   Disponibilité de matière grasse en quantité (g/personne/jour)  \
0                                                 0.20
1                                                 0.01
2                                                 0.01
3                                                 0.00
4                                                 0.02

   Disponibilité de protéines en quantité (g/personne/jour)  \
0                                                 0.77
1                                                 0.02
2                                                 0.03
3                                                 0.00
4                                                 0.05

   Disponibilité intérieure  Exportations - Quantité  Importations - Quantité  \
0               53000000.0                       0.0                       0.0
1               41000000.0                 2000000.0                40000000.0
2                2000000.0                       0.0                 2000000.0
3                      0.0                       0.0                       0.0
4               82000000.0                       0.0                82000000.0

   Nourriture     Pertes  Production  Semences  Traitement  \
0  53000000.0        0.0  53000000.0       0.0         0.0
1  39000000.0  2000000.0   3000000.0       0.0         0.0
2   2000000.0        0.0         0.0       0.0         0.0
3         0.0        0.0         0.0       0.0         0.0
4  82000000.0        0.0         0.0       0.0         0.0

   Variation de stock   Année  Population
0                 0.0  2017.0  36296113.0
1                 0.0  2017.0  36296113.0
2                 0.0  2017.0  36296113.0
3                 0.0  2017.0  36296113.0
4                 0.0  2017.0  36296113.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15605 entries, 0 to 15604
Data columns (total 20 columns):
 #   Column                                                      Non-Null
Count  Dtype
---  ------
--------------  -----
 0   Zone                                                        15605 non-
null  object
 1   Produit                                                     15605 non-
null  object
 2   Origine                                                     15605 non-
```

```
null  object
 3   Aliments pour animaux                                          15605 non-
null  float64
 4   Autres Utilisations                                            15605 non-
null  float64
 5   Disponibilité alimentaire (Kcal/personne/jour)                 15605 non-
null  float64
 6   Disponibilité alimentaire en quantité (kg/personne/an)         15605 non-
null  float64
 7   Disponibilité de matière grasse en quantité (g/personne/jour)  15605 non-
null  float64
 8   Disponibilité de protéines en quantité (g/personne/jour)       15605 non-
null  float64
 9   Disponibilité intérieure                                       15605 non-
null  float64
 10  Exportations - Quantité                                        15605 non-
null  float64
 11  Importations - Quantité                                        15605 non-
null  float64
 12  Nourriture                                                     15605 non-
null  float64
 13  Pertes                                                         15605 non-
null  float64
 14  Production                                                     15605 non-
null  float64
 15  Semences                                                       15605 non-
null  float64
 16  Traitement                                                     15605 non-
null  float64
 17  Variation de stock                                             15605 non-
null  float64
 18  Année                                                          15416 non-
null  float64
 19  Population                                                     15416 non-
null  float64
dtypes: float64(17), object(3)
memory usage: 2.4+ MB
None
```

[61]:
```
# Création de la colonne dispo_kcal (kcal disponibles totales par pays en 2017)
dispo_pop["dispo_kcal"] = dispo_pop["Disponibilité alimentaire (Kcal/personne/
 ↪jour)"] \
                          * dispo_pop["Population"] * 365

# Besoin annuel par personne (kcal)
besoin_annuel = 2500 * 365  # = 912500 kcal
```

[62]:
```
#Calcul du nombre d'humains pouvant être nourris

# Calcul du nombre théorique d'humains pouvant être nourris
dispo_pop["humains_nourris"] = dispo_pop["dispo_kcal"] / besoin_annuel

# Vérification
print(dispo_pop[["Zone", "Population", "Disponibilité alimentaire (Kcal/
 ↪personne/jour)", "dispo_kcal", "humains_nourris"]].head(10))

# Total mondial
total_humains_nourris = dispo_pop["humains_nourris"].sum()
print("Nombre théorique d'humains pouvant être nourris en 2017 :",␣
 ↪int(total_humains_nourris))
```

```
          Zone  Population  Disponibilité alimentaire (Kcal/personne/jour)  \
0  Afghanistan  36296113.0                                            5.0
1  Afghanistan  36296113.0                                            1.0
2  Afghanistan  36296113.0                                            1.0
3  Afghanistan  36296113.0                                            0.0
4  Afghanistan  36296113.0                                            4.0
5  Afghanistan  36296113.0                                           23.0
6  Afghanistan  36296113.0                                            0.0
7  Afghanistan  36296113.0                                         1369.0
8  Afghanistan  36296113.0                                            0.0
9  Afghanistan  36296113.0                                            0.0

     dispo_kcal  humains_nourris
0  6.624041e+10     7.259223e+04
1  1.324808e+10     1.451845e+04
2  1.324808e+10     1.451845e+04
3  0.000000e+00     0.000000e+00
4  5.299232e+10     5.807378e+04
5  3.047059e+11     3.339242e+05
6  0.000000e+00     0.000000e+00
7  1.813662e+13     1.987575e+07
8  0.000000e+00     0.000000e+00
9  0.000000e+00     0.000000e+00
Nombre théorique d'humains pouvant être nourris en 2017 : 8367593850
```

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

[63]:
```
#Transfert des données avec les végétaux dans un nouveau dataframe
import pandas as pd

# 1) Filtre de la population sur 2017
pop_2017 = population[population["Année"] == 2017].copy()

# 2) Joindre la population au fichier de disponibilité 2017
```

```python
dispo_pop = pd.merge(
    dispo_alimentaire,  # DataFrame 2017 selon le lexique
    pop_2017[["Zone", "Population"]],
    on="Zone",
    how="left"
)

# 3) Création du DataFrame végétal
# Ajuste le nom exact de la colonne si nécessaire
dispo_veg = dispo_pop[dispo_pop["Origine"].str.lower().str.contains("végét")].
 ↪copy()

# (Option) Nettoyage des NaN sur les colonnes clés
cols_kcal = ["Disponibilité alimentaire (Kcal/personne/jour)", "Population"]
dispo_veg[cols_kcal] = dispo_veg[cols_kcal].fillna(0)

print(dispo_veg.head())
print(dispo_veg.info())
```

```
Empty DataFrame
Columns: [Zone, Produit, Origine, Aliments pour animaux, Autres Utilisations,
Disponibilité alimentaire (Kcal/personne/jour), Disponibilité alimentaire en
quantité (kg/personne/an), Disponibilité de matière grasse en quantité
(g/personne/jour), Disponibilité de protéines en quantité (g/personne/jour),
Disponibilité intérieure, Exportations - Quantité, Importations - Quantité,
Nourriture, Pertes, Production, Semences, Traitement, Variation de stock,
Population]
Index: []
<class 'pandas.core.frame.DataFrame'>
Index: 0 entries
Data columns (total 19 columns):
 #   Column                                                  Non-Null
Count  Dtype
---  ------
--------------  -----
 0   Zone                                                    0 non-null
object
 1   Produit                                                 0 non-null
object
 2   Origine                                                 0 non-null
object
 3   Aliments pour animaux                                   0 non-null
float64
 4   Autres Utilisations                                     0 non-null
float64
 5   Disponibilité alimentaire (Kcal/personne/jour)          0 non-null
float64
 6   Disponibilité alimentaire en quantité (kg/personne/an)  0 non-null
```

```
float64
 7   Disponibilité de matière grasse en quantité (g/personne/jour)  0 non-null
float64
 8   Disponibilité de protéines en quantité (g/personne/jour)       0 non-null
float64
 9   Disponibilité intérieure                                       0 non-null
float64
 10  Exportations - Quantité                                        0 non-null
float64
 11  Importations - Quantité                                        0 non-null
float64
 12  Nourriture                                                     0 non-null
float64
 13  Pertes                                                         0 non-null
float64
 14  Production                                                     0 non-null
float64
 15  Semences                                                       0 non-null
float64
 16  Traitement                                                     0 non-null
float64
 17  Variation de stock                                             0 non-null
float64
 18  Population                                                     0 non-null
float64
dtypes: float64(16), object(3)
memory usage: 0.0+ bytes
None
```

[67]:
```python
###Calcul du nombre de kcal disponible pour les végétaux

# Je fais un filtre uniquement sur les produits d'origine végétale
dispo_veg = dispo_pop[dispo_pop["Origine"].str.lower() == "vegetale"].copy()
#Transfert des données avec les végétaux dans un nouveau dataframe
# Création de la colonne kcal disponibles annuelles (végétal)
dispo_veg["dispo_kcal_veg"] = (
    dispo_veg["Disponibilité alimentaire (Kcal/personne/jour)"]
    * dispo_veg["Population"]
    * 365

)
print(dispo_veg)
# Besoin annuel par personne
BESOIN_ANNUEL_KCAL = 2500 * 365  # 912 500 kcal/an/personne
```

```
               Zone                Produit   Origine  Aliments pour animaux  \
1       Afghanistan        Agrumes, Autres  vegetale                    0.0
2       Afghanistan  Aliments pour enfants  vegetale                    0.0
```

3	Afghanistan	Ananas	vegetale	0.0
4	Afghanistan	Bananes	vegetale	0.0
6	Afghanistan	Bière	vegetale	0.0
…	…	…	…	…
15595	Îles Salomon	Sésame	vegetale	0.0
15596	Îles Salomon	Thé	vegetale	0.0
15597	Îles Salomon	Tomates	vegetale	0.0
15603	Îles Salomon	Vin	vegetale	0.0
15604	Îles Salomon	Épices, Autres	vegetale	0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
1	0.0	1.0
2	0.0	1.0
3	0.0	0.0
4	0.0	4.0
6	0.0	0.0
…	…	…
15595	0.0	0.0
15596	0.0	0.0
15597	0.0	0.0
15603	0.0	0.0
15604	0.0	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
1	1.29
2	0.06
3	0.00
4	2.70
6	0.09
…	…
15595	0.01
15596	0.17
15597	0.07
15603	0.07
15604	0.48

	Disponibilité de matière grasse en quantité (g/personne/jour) \
1	0.01
2	0.01
3	0.00
4	0.02
6	0.00
…	…
15595	0.00
15596	0.00
15597	0.00
15603	0.00
15604	0.21

	Disponibilité de protéines en quantité (g/personne/jour) \
1	0.02
2	0.03
3	0.00
4	0.05
6	0.00
…	…
15595	0.00
15596	0.04
15597	0.00
15603	0.00
15604	0.15

	Disponibilité intérieure	Exportations - Quantité \
1	41000000.0	2000000.0
2	2000000.0	0.0
3	0.0	0.0
4	82000000.0	0.0
6	3000000.0	0.0
…	…	…
15595	0.0	0.0
15596	0.0	0.0
15597	0.0	0.0
15603	0.0	0.0
15604	0.0	0.0

	Importations - Quantité	Nourriture	Pertes	Production	Semences \
1	40000000.0	39000000.0	2000000.0	3000000.0	0.0
2	2000000.0	2000000.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	82000000.0	82000000.0	0.0	0.0	0.0
6	3000000.0	3000000.0	0.0	0.0	0.0
…	…	…	…	…	…
15595	0.0	0.0	0.0	0.0	0.0
15596	0.0	0.0	0.0	0.0	0.0
15597	0.0	0.0	0.0	0.0	0.0
15603	0.0	0.0	0.0	0.0	0.0
15604	0.0	0.0	0.0	0.0	0.0

	Traitement	Variation de stock	Population	dispo_kcal_veg
1	0.0	0.0	36296113.0	1.324808e+10
2	0.0	0.0	36296113.0	1.324808e+10
3	0.0	0.0	36296113.0	0.000000e+00
4	0.0	0.0	36296113.0	5.299232e+10
6	0.0	0.0	36296113.0	0.000000e+00
…	…	…	…	…
15595	0.0	0.0	636039.0	0.000000e+00

```
15596          0.0                  0.0    636039.0    0.000000e+00
15597          0.0                  0.0    636039.0    0.000000e+00
15603          0.0                  0.0    636039.0    0.000000e+00
15604          0.0                  0.0    636039.0    9.286169e+08

[11896 rows x 20 columns]
```

[68]:
```
#Calcul du nombre d'humains pouvant être nourris avec les végétaux
# Nombre d'humains nourris par ligne
dispo_veg["humains_nourris_veg"] = dispo_veg["dispo_kcal_veg"] /␣
↪BESOIN_ANNUEL_KCAL
pays_veg = dispo_veg.groupby("Zone", as_index=False).agg({
    "dispo_kcal_veg": "sum",
    "humains_nourris_veg": "sum"
})
# Total mondial
total_humains_veg_monde = pays_veg["humains_nourris_veg"].sum()

print(pays_veg.head(10))
print(f"Nombre théorique d'humains nourris avec les produits végétaux (monde,␣
↪2017): {int(total_humains_veg_monde):,}")
```

```
                 Zone  dispo_kcal_veg  humains_nourris_veg
0         Afghanistan    2.478716e+13         2.716401e+07
1      Afrique du Sud    5.270808e+13         5.776228e+07
2             Albanie    2.319146e+12         2.541530e+06
3             Algérie    4.403706e+13         4.825979e+07
4           Allemagne    7.424916e+13         8.136894e+07
5              Angola    2.417141e+13         2.648921e+07
6  Antigua-et-Barbuda    5.694785e+10         6.240860e+04
7     Arabie saoudite    3.305616e+13         3.622593e+07
8           Argentine    3.590697e+13         3.935010e+07
9             Arménie    2.300176e+12         2.520741e+06
Nombre théorique d'humains nourris avec les produits végétaux (monde, 2017):
6,904,305,684
```

3.4 - Utilisation de la disponibilité intérieure

[73]:
```
### Calcul de la disponibilité Totale

# Filtrer l'année 2017 dans les deux jeux
pop_2017 = population[population["Année"] == 2017][["Zone", "Population"]].
↪drop_duplicates()

if "Année" in dispo_alimentaire.columns:
    dispo_2017 = dispo_alimentaire[dispo_alimentaire["Année"] == 2017].copy()
else:
    dispo_2017 = dispo_alimentaire.copy()
```

```python
# Colonnes d'utilisation en kg (totaux par ligne)
cols_utilisation = [
    "Nourriture",
    "Aliments pour animaux",
    "Semences",
    "Pertes",
    "Traitement",
    "Autres Utilisations"
]

# Conversion en numérique
for c in cols_utilisation:
    dispo_2017[c] = pd.to_numeric(dispo_2017[c], errors="coerce").fillna(0)

# Agrégation monde: somme simple des usages (aucune population, aucun kcal/hab)
utilisation_monde = dispo_2017[cols_utilisation].sum()

# Disponibilité intérieure totale (somme de toutes les utilisations)
dispo_int_totale = utilisation_monde.sum()

# Parts en %
parts = (utilisation_monde / dispo_int_totale) * 100

# Affichage
print("Totaux monde (kg):")
for c in cols_utilisation:
    print(f"- {c}: {utilisation_monde[c]:,.0f} kg ({parts[c]:.2f}%)")

print(f"\nDisponibilité intérieure totale (monde, 2017): {dispo_int_totale:,.
↪0f} kg")
```

```
Totaux monde (kg):
- Nourriture: 4,876,258,000,000 kg (54.22%)
- Aliments pour animaux: 1,304,245,000,000 kg (14.50%)
- Semences: 154,681,000,000 kg (1.72%)
- Pertes: 453,698,000,000 kg (5.04%)
- Traitement: 2,204,687,000,000 kg (24.51%)
- Autres Utilisations: 865,023 kg (0.00%)

Disponibilité intérieure totale (monde, 2017): 8,993,569,865,023 kg
```

[94]:
```python
### création d'une boucle for pour afficher les différentes valeurs
### en fonction des colonnes Aliments pour animaux, Pertes, Nourriture

# Colonnes d'utilisation ciblées
cols_util_1 = [
```

```
    "Aliments pour animaux",
    "Pertes",
    "Nourriture",
]

# Conversion en numérique
for c in cols_util_1:
    dispo_pop[c] = pd.to_numeric(dispo_pop[c], errors="coerce").fillna(0)

# Agrégation mondiale par poste (somme simple des colonnes ciblées)
utilisation_monde = dispo_pop[cols_util_1].sum()

# Disponibilité intérieure totale (toutes les colonnes)
cols_utilisation = [
    "Nourriture",
    "Aliments pour animaux",
    "Pertes",
    "Semences",
    "Traitement",
    "Autres Utilisations"
]
dispo_int_totale = dispo_pop[cols_utilisation].sum().sum()

# Parts en % par rapport à la disponibilité intérieure totale
parts = (utilisation_monde / dispo_int_totale) * 100

# Affichage
for c in cols_util_1:
    print(f"- {c}: {utilisation_monde[c]:,.0f} kg ({parts[c]:.2f}%)")

print(f"\nDisponibilité intérieure totale (monde, 2017): {dispo_int_totale:,.
 ↪0f} kg")
```

```
- Aliments pour animaux: 1,304,245,000,000 kg (14.50%)
- Pertes: 453,698,000,000 kg (5.04%)
- Nourriture: 4,876,258,000,000 kg (54.22%)

Disponibilité intérieure totale (monde, 2017): 8,993,569,865,023 kg
```

[93]:
```
# Calcul des autres valeurs

# Colonnes d'utilisation ciblées
cols_util_2 = [
    "Semences",
    "Traitement",
    "Autres Utilisations"
]
```

```python
# Conversion en numérique
for c in cols_util_2:
    dispo_pop[c] = pd.to_numeric(dispo_pop[c], errors="coerce").fillna(0)

# Agrégation mondiale pour les colonnes ciblées (somme simple)
utilisation_monde = dispo_pop[cols_util_2].sum()

# Disponibilité intérieure totale (toutes les colonnes)
cols_utilisation = [
    "Nourriture",
    "Aliments pour animaux",
    "Pertes",
    "Semences",
    "Traitement",
    "Autres Utilisations"
]
dispo_int_totale = dispo_pop[cols_utilisation].sum().sum()

# Parts en % par rapport à la disponibilité intérieure totale
parts = (utilisation_monde / dispo_int_totale) * 100

# Affichage
for c in cols_util_2:
    print(f"- {c}: {utilisation_monde[c]:,.0f} kg ({parts[c]:.2f}%)")

print(f"\nDisponibilité intérieure totale (monde, 2017): {dispo_int_totale:,.
 ↪0f} kg")
```

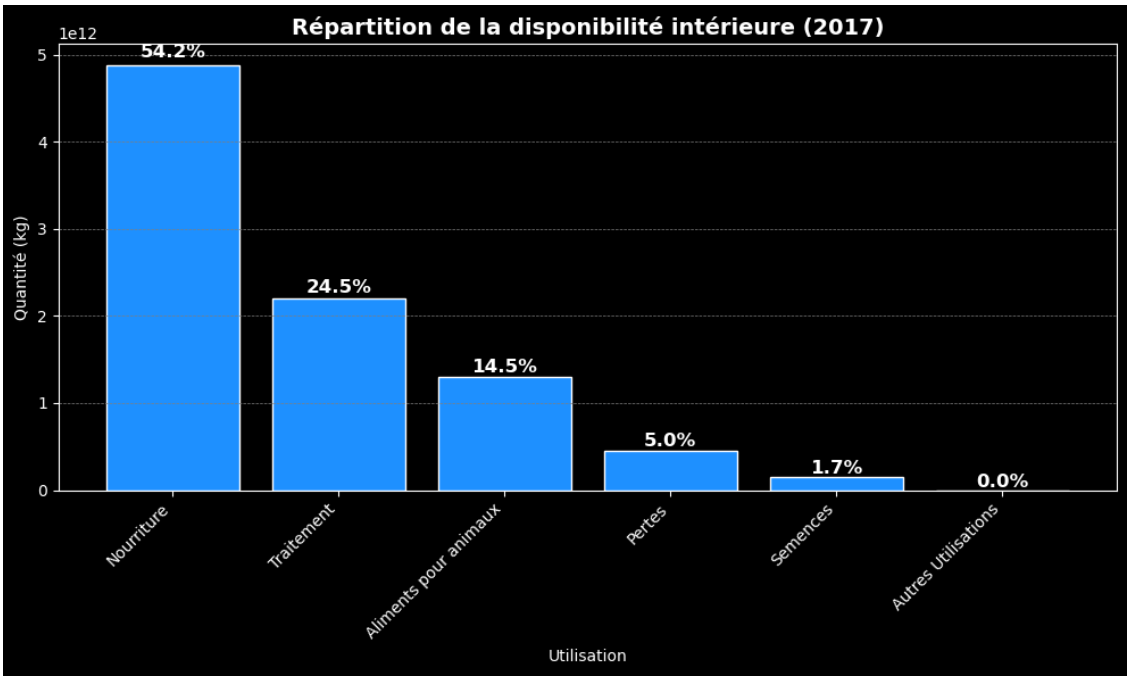
```
- Semences: 154,681,000,000 kg (1.72%)
- Traitement: 2,204,687,000,000 kg (24.51%)
- Autres Utilisations: 865,023 kg (0.00%)

Disponibilité intérieure totale (monde, 2017): 8,993,569,865,023 kg
```

[345]:
```python
#print(dispo_alimentaire.columns.tolist())
```

[95]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Activer un style sombre
plt.style.use("dark_background")

# Colonnes d'utilisation
cols_utilisation = [
    "Aliments pour animaux",
    "Semences",
    "Pertes",
```

```
    "Nourriture",
    "Traitement",
    "Autres Utilisations"
]

# 1) Agrégation mondiale
utilisation_monde = dispo_alimentaire[cols_utilisation].sum()

# 2) Calcul des parts en %
parts = (utilisation_monde / utilisation_monde.sum()) * 100

# 3) Tri par ordre décroissant
utilisation_monde_sorted = utilisation_monde.sort_values(ascending=False)
parts_sorted = parts.loc[utilisation_monde_sorted.index]

# 4) Affichage des valeurs triées
for col in utilisation_monde_sorted.index:
    print(f"{col}: {utilisation_monde_sorted[col]:,.0f} kg ({parts_sorted[col]:.
 ↪2f}%)")

# 5) Graphique en diagramme à barres trié avec fond sombre
fig, ax = plt.subplots(figsize=(10,6))
bars = ax.bar(utilisation_monde_sorted.index, utilisation_monde_sorted.values,
              color="dodgerblue", edgecolor="white")

ax.set_title("Répartition de la disponibilité intérieure (2017)", fontsize=14,␣
 ↪fontweight="bold")
ax.set_ylabel("Quantité (kg)")
ax.set_xlabel("Utilisation")
ax.set_xticks(range(len(utilisation_monde_sorted)))
ax.set_xticklabels(utilisation_monde_sorted.index, rotation=45, ha="right")
ax.grid(axis="y", color="grey", linewidth=0.5, linestyle="--")

# 6) Affichage des pourcentages au-dessus des barres en blanc et gras
for i, val in enumerate(utilisation_monde_sorted.values):
    ax.text(
        i, val + val*0.01,
        f"{parts_sorted.iloc[i]:.1f}%",
        ha="center", va="bottom",
        fontsize=12,
        color="white",
        fontweight="bold"
    )

plt.tight_layout()
plt.show()
```

```
Nourriture: 4,876,258,000,000 kg (54.22%)
Traitement: 2,204,687,000,000 kg (24.51%)
Aliments pour animaux: 1,304,245,000,000 kg (14.50%)
Pertes: 453,698,000,000 kg (5.04%)
Semences: 154,681,000,000 kg (1.72%)
Autres Utilisations: 865,023 kg (0.00%)
```



3.5 - Utilisation des céréales

[96]:
```python
# Liste des colonnes d'utilisation de la disponibilité intérieure
# Création d'une liste avec toutes les variables

# Liste des colonnes d'utilisation de la disponibilité intérieure
cols_utilisation = [
    "Aliments pour animaux",
    "Semences",
    "Pertes",
    "Nourriture",
    "Traitement",
    "Autres Utilisations"
]

# Liste des céréales selon la FAO
liste_cereales = [
    "Blé",
    "Riz (Eq Blanchi)",
```

```
    "Orge",
    "Maïs",
    "Seigle",
    "Avoine",
    "Millet",
    "Sorgho",
    "Céréales, Autres"
]

# Vérification des listes
print("Colonnes retenues :", cols_utilisation)
print("Produits céréaliers :", liste_cereales)
```

```
Colonnes retenues : ['Aliments pour animaux', 'Semences', 'Pertes',
'Nourriture', 'Traitement', 'Autres Utilisations']
Produits céréaliers : ['Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Seigle',
'Avoine', 'Millet', 'Sorgho', 'Céréales, Autres']
```

[97]:
```
###Création d'un dataframe avec les informations uniquement pour ces céréales
# Filtre unique des céréales
cereales = dispo_alimentaire[dispo_alimentaire["Produit"].isin(liste_cereales)]

# Nouveau DataFrame avec uniquement les colonnes utiles
df_cereales = cereales[["Zone", "Produit"] + cols_utilisation]

print(df_cereales.head())
```

```
           Zone           Produit  Aliments pour animaux     Semences  \
7   Afghanistan               Blé                    0.0  322000000.0
12  Afghanistan  Céréales, Autres                    0.0          0.0
32  Afghanistan              Maïs            200000000.0    5000000.0
34  Afghanistan            Millet                    0.0          0.0
40  Afghanistan              Orge            360000000.0   22000000.0

         Pertes    Nourriture  Traitement  Autres Utilisations
7   775000000.0  4.895000e+09         0.0                  0.0
12          0.0  0.000000e+00         0.0                  0.0
32   31000000.0  7.600000e+07         0.0                  0.0
34    1000000.0  1.200000e+07         0.0                  0.0
40   52000000.0  8.900000e+07         0.0                  0.0
```

[98]:
```
### Affichage de la proportion d'alimentation animale

# Agrégation mondiale pour les céréales uniquement
utilisation_cereales = df_cereales[cols_utilisation].sum()

# Disponibilité intérieure totale des céréales
dispo_int_cereales = utilisation_cereales.sum()
```

```
# Proportion de l'alimentation animale
part_animaux = (utilisation_cereales["Aliments pour animaux"] /␣
↪dispo_int_cereales) * 100

# Affichage
print(f"Alimentation animale (céréales, monde, 2017):␣
↪{utilisation_cereales['Aliments pour animaux']:,.0f} kg")
print(f"Part de l'alimentation animale dans la disponibilité intérieure totale␣
↪des céréales: {part_animaux:.2f}%")
```

```
Alimentation animale (céréales, monde, 2017): 873,535,000,000 kg
Part de l'alimentation animale dans la disponibilité intérieure totale des
céréales: 40.20%
```

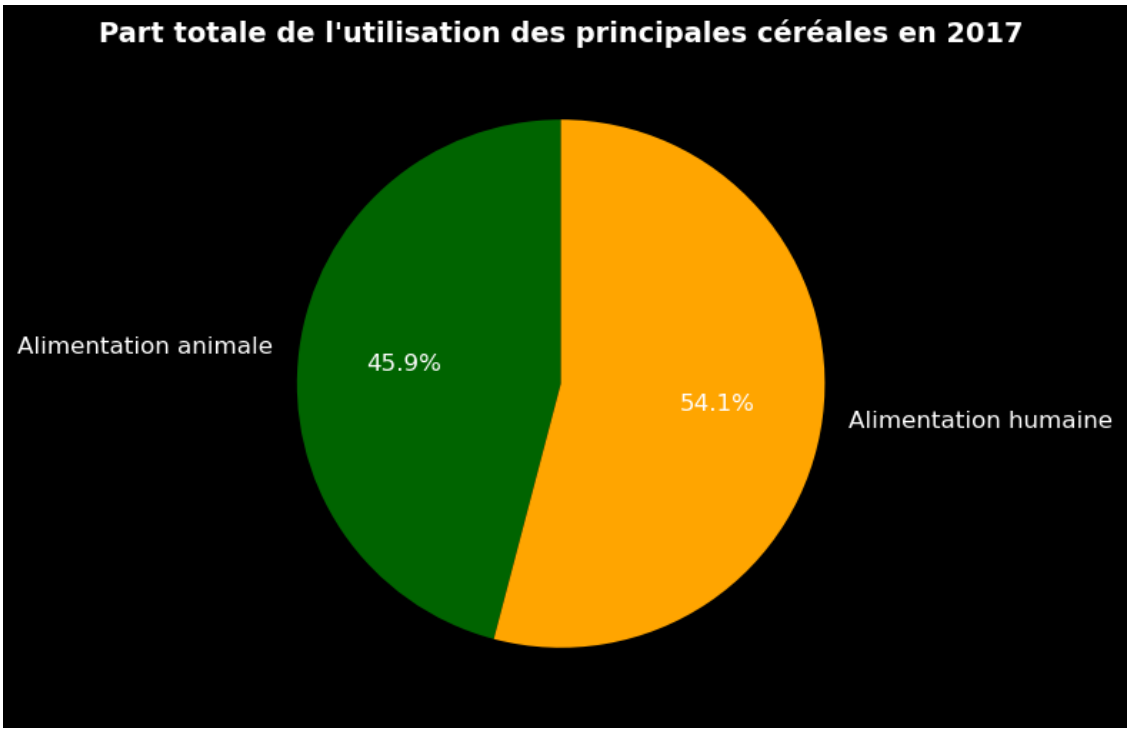
[ ]:
```
### DANS LE NOTEBOOK INITIALE LA QUESTION SUR LA PROPORTION D'ALIMENTATION␣
↪ANIMALE EST RÉPÉTÉE SUR DEUX CELLULES
### C'EST PEUT-ÊTRE INVOLONTAIRE MAIS ÇA INDUIT EN ERREUR.
```

[99]:
```
### Pour besoin de la suite, je décide de calculer la part de l'alimentation␣
↪humaine
### Affichage de la proportion d'alimentation humaine ("Nourriture")

# Agrégation mondiale pour les céréales uniquement
utilisation_cereales = df_cereales[cols_utilisation].sum()

# Disponibilité intérieure totale des céréales
dispo_int_totale_cereales = utilisation_cereales.sum()

# Proportion de l'alimentation humaine ("Nourriture")
part_humaine = (utilisation_cereales["Nourriture"] / dispo_int_totale_cereales)␣
↪* 100

# Affichage
print(f"Nourriture (céréales, monde, 2017): {utilisation_cereales['Nourriture']:
↪,.0f} kg")
print(f"Part de la nourriture dans la disponibilité intérieure totale des␣
↪céréales: {part_humaine:.2f}%")
```

```
Nourriture (céréales, monde, 2017): 1,029,010,000,000 kg
Part de la nourriture dans la disponibilité intérieure totale des céréales:
47.36%
```

[106]:
```
import matplotlib.pyplot as plt

### Camembert des parts alimentation animale vs humaine

# Agrégation mondiale pour les céréales uniquement
```

```python
utilisation_cereales = df_cereales[cols_utilisation].sum()

# Disponibilité intérieure totale des céréales
dispo_int_totale_cereales = utilisation_cereales.sum()

# Calcul des parts
part_animaux = (utilisation_cereales["Aliments pour animaux"] /␣
 ↪dispo_int_totale_cereales) * 100
part_humaine = (utilisation_cereales["Nourriture"] / dispo_int_totale_cereales)␣
 ↪* 100

# Données pour le camembert
labels = ["Alimentation animale", "Alimentation humaine"]
sizes = [part_animaux, part_humaine]
colors = ["darkgreen", "orange"]

# Style sombre
plt.style.use("dark_background")

# Création du camembert
fig, ax = plt.subplots(figsize=(6,6))
wedges, texts, autotexts = ax.pie(
    sizes,
    labels=labels,
    autopct="%.1f%%",
    startangle=90,
    colors=colors,
    textprops={"color":"white", "fontsize":12}
)

# Titre
ax.set_title("Part totale de l'utilisation des principales céréales en 2017",␣
 ↪fontsize=14, fontweight="bold")

plt.show()
```

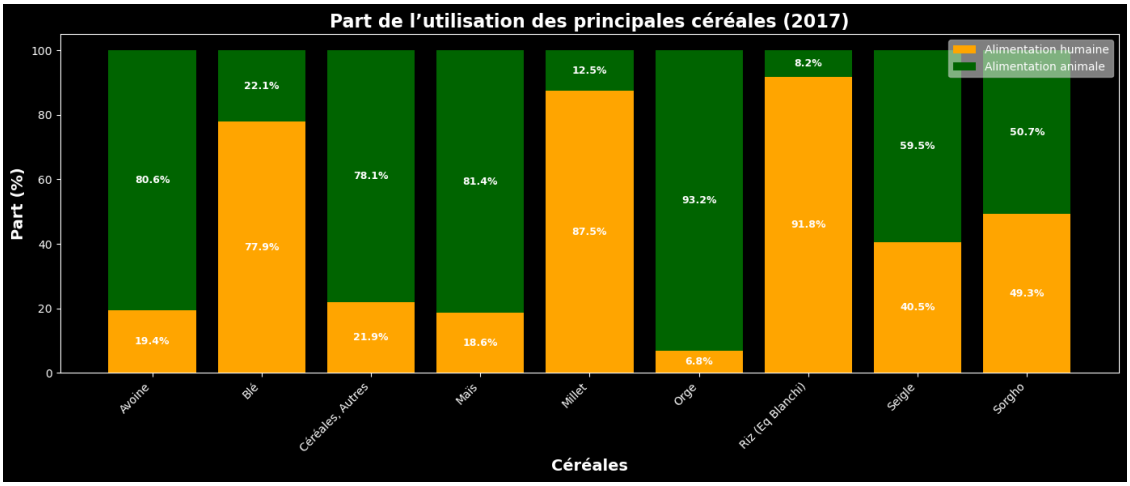


[ ]:
```
### DANS LE POWERPOINT, JULIEN S'INTERROGE S'IL DEVAIT PRESENTER LA REPARTITION␣
↪PAR CEREALE OU LE TOTAL.
#JE CHOISIS DE REPRESENTER LES DEUX DANS LE POWERPOINT
```

[131]:
```
# Pour le PowerPoint
import matplotlib.pyplot as plt

# Colonnes d'utilisation
cols_utilisation = ["Nourriture", "Aliments pour animaux"]

# Liste des céréales selon la FAO
liste_cereales = [
    "Blé",
    "Riz (Eq Blanchi)",
    "Orge",
    "Maïs",
    "Seigle",
    "Avoine",
    "Millet",
    "Sorgho",
    "Céréales, Autres"
]

# 1) Filtrer uniquement les céréales
```

```python
cereales = dispo_alimentaire[dispo_alimentaire["Produit"].isin(liste_cereales)]

# 2) Agrégation par produit
utilisation_par_cereale = cereales.groupby("Produit")[cols_utilisation].sum()

# 3) Calcul des parts en %
utilisation_par_cereale_pct = utilisation_par_cereale.
 ↪div(utilisation_par_cereale.sum(axis=1), axis=0) * 100

# 4) Graphique en barres empilées
plt.style.use("dark_background")  # fond sombre
fig, ax = plt.subplots(figsize=(14,6))

bottom = None
colors = ["orange", "darkgreen"]  # vert pour humain, orange pour animal

for i, col in enumerate(cols_utilisation):
    values = utilisation_par_cereale_pct[col]
    ax.bar(utilisation_par_cereale_pct.index, values, bottom=bottom, label=col,␣
 ↪color=colors[i])

    # Ajouter les valeurs exactes (%) sur chaque segment
    for j, val in enumerate(values):
        if bottom is None:
            y = val / 2
        else:
            y = bottom.iloc[j] + val / 2
        ax.text(j, y, f"{val:.1f}%", ha="center", va="center", color="white",␣
 ↪fontsize=9, fontweight="bold")

    # Mettre à jour le "bottom" pour empiler
    if bottom is None:
        bottom = values
    else:
        bottom = bottom + values

# Mise en forme
ax.set_title("Part de l'utilisation des principales céréales (2017)",␣
 ↪fontsize=16, fontweight="bold")
ax.set_ylabel("Part (%)", fontsize=14, fontweight="bold")
ax.set_xlabel("Céréales", fontsize=14, fontweight="bold")

# Légende avec fond blanc transparent
legend = ax.legend(
    ["Alimentation humaine", "Alimentation animale"],
    loc="upper right",
    frameon=True,
```

```
    facecolor="white",
    framealpha=0.5
)

plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

[121]:
```
# Création de la colonne proportion par pays
df_2017["proportion_sous_nutrition"] = (df_2017["sous_nutrition"] /␣
↪df_2017["Population"]) * 100

# Vérification rapide
print(df_2017[["Zone", "sous_nutrition", "Population",␣
↪"proportion_sous_nutrition"]].head())
```

```
            Zone  sous_nutrition  Population  proportion_sous_nutrition
0    Afghanistan      11100000.0  36296113.0                  30.581787
1  Afrique du Sud       3300000.0  57009756.0                   5.788483
2        Albanie        100000.0   2884169.0                   3.467203
3        Algérie       1200000.0  41389189.0                   2.899308
4      Allemagne             0.0  82658409.0                   0.000000
```

[122]:
```
#affichage après trie des 10 pires pays
# Trier par proportion décroissante
df_sorted = df_2017.sort_values("proportion_sous_nutrition", ascending=False)

# Afficher les 10 premiers pays
```

```
print("Les 10 pays avec la proportion la plus forte de personnes en␣
↪sous-nutrition en 2017 :")
for _, r in df_sorted.head(10).iterrows():
    print(f"{r['Zone']}: {r['sous_nutrition']:,.0f} personnes␣
↪({r['proportion_sous_nutrition']:.2f} %)")
```

```
Les 10 pays avec la proportion la plus forte de personnes en sous-nutrition en
2017 :
Dominique: 100,000 personnes (139.94 %)
Saint-Vincent-et-les Grenadines: 100,000 personnes (91.05 %)
Kiribati: 100,000 personnes (87.60 %)
Haïti: 5,400,000 personnes (49.17 %)
Sao Tomé-et-Principe: 100,000 personnes (48.29 %)
République populaire démocratique de Corée: 12,200,000 personnes (47.98 %)
Madagascar: 11,000,000 personnes (43.02 %)
Tchad: 6,100,000 personnes (40.62 %)
Libéria: 1,800,000 personnes (38.28 %)
Rwanda: 4,400,000 personnes (36.72 %)
```

[ ]:
```
### DES PROPORTIONS DEPASSENT LES 100%
#Pour rester cohérent, je vais faire un filtre pour exclure les etats avec des␣
↪population < 1 000 000 d'hab
#car il me semble que le chiffres de la FAO sont biasés du fait des arrondis et␣
↪aussi par la multiplication faite plus haut
### Au cas où mon raisonnement ne répondrait aux attentes parce qu'il faut␣
↪absolument se fier au ficher,
#vous pourrez vous reporter uniquement au calcul précédent.
```

[128]:
```
df_filtered = df_2017[df_2017["Population"] > 1_000_000]
df_sorted = df_filtered.sort_values("proportion_sous_nutrition",␣
↪ascending=False)

print("Liste des 10 pays où la proportion de personnes en état de␣
↪sous-nutrition est la plus forte en 2017")
for _, r in df_sorted.head(10).iterrows():
    print(f"{r['Zone']}: {r['sous_nutrition']:,.0f} personnes␣
↪({r['proportion_sous_nutrition']:.2f} %)")
```

```
Liste des 10 pays où la proportion de personnes en état de sous-nutrition est la
plus forte en 2017
Haïti: 5,400,000 personnes (49.17 %)
République populaire démocratique de Corée: 12,200,000 personnes (47.98 %)
Madagascar: 11,000,000 personnes (43.02 %)
Tchad: 6,100,000 personnes (40.62 %)
Libéria: 1,800,000 personnes (38.28 %)
Rwanda: 4,400,000 personnes (36.72 %)
Mozambique: 9,600,000 personnes (33.51 %)
Lesotho: 700,000 personnes (33.47 %)
```

```
Timor-Leste: 400,000 personnes (32.17 %)
Venezuela (République bolivarienne du): 9,100,000 personnes (30.95 %)
```

[127]:
```python
# Sélectionner et renommer les colonnes pour le tableau
tableau = df_sorted.head(10)[["Zone", "sous_nutrition", "Population",␣
↪"proportion_sous_nutrition"]]
tableau = tableau.rename(columns={
    "Zone": "Pays",
    "sous_nutrition": "Personnes sous-nutrition",
    "Population": "Population totale",
    "proportion_sous_nutrition": "Proportion (%)"
})

# Afficher le tableau
tableau
```

[127]:
```
                                          Pays  Personnes sous-nutrition  \
78                                       Haïti                 5400000.0
157  République populaire démocratique de Corée                12200000.0
108                                 Madagascar                11000000.0
183                                      Tchad                 6100000.0
103                                   Libéria                 1800000.0
161                                    Rwanda                 4400000.0
121                                 Mozambique                 9600000.0
100                                    Lesotho                  700000.0
186                                 Timor-Leste                  400000.0
198      Venezuela (République bolivarienne du)                 9100000.0

     Population totale  Proportion (%)
78          10982366.0       49.169733
157         25429825.0       47.975163
108         25570512.0       43.018302
183         15016753.0       40.621298
103          4702226.0       38.279742
161         11980961.0       36.724934
121         28649018.0       33.509002
100          2091534.0       33.468258
186          1243258.0       32.173531
198         29402484.0       30.949766
```

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

[129]:
```python
#calcul du total de l'aide alimentaire par pays
# Agrégation de l'aide alimentaire par pays sur toute la période disponible␣
↪(depuis 2013)
aide_totale_par_pays = aide_alimentaire.groupby("Zone")["Valeur"].sum()

# Affichage des premiers résultats
```

```
print(aide_totale_par_pays.head())
```

```
Zone
Afghanistan    185452
Algérie         81114
Angola           5014
Bangladesh     348188
Bhoutan          2666
Name: Valeur, dtype: int64
```

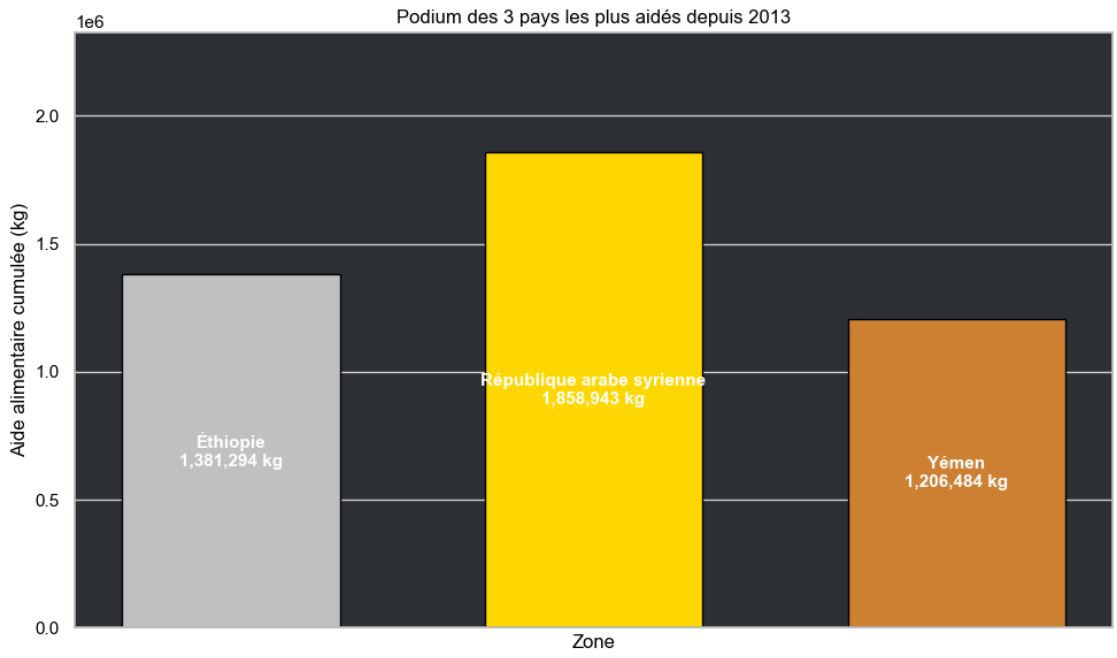
[130]:
```
#affichage après trie des 10 pays qui ont bénéficié le plus de l'aide␣
↪alimentaire
# Tri décroissant des pays selon l'aide reçue
top_10_aide = aide_totale_par_pays.sort_values(ascending=False).head(10)

# Affichage
print("Top 10 des pays ayant reçu le plus d'aide alimentaire depuis 2013 :")
print(top_10_aide)
```

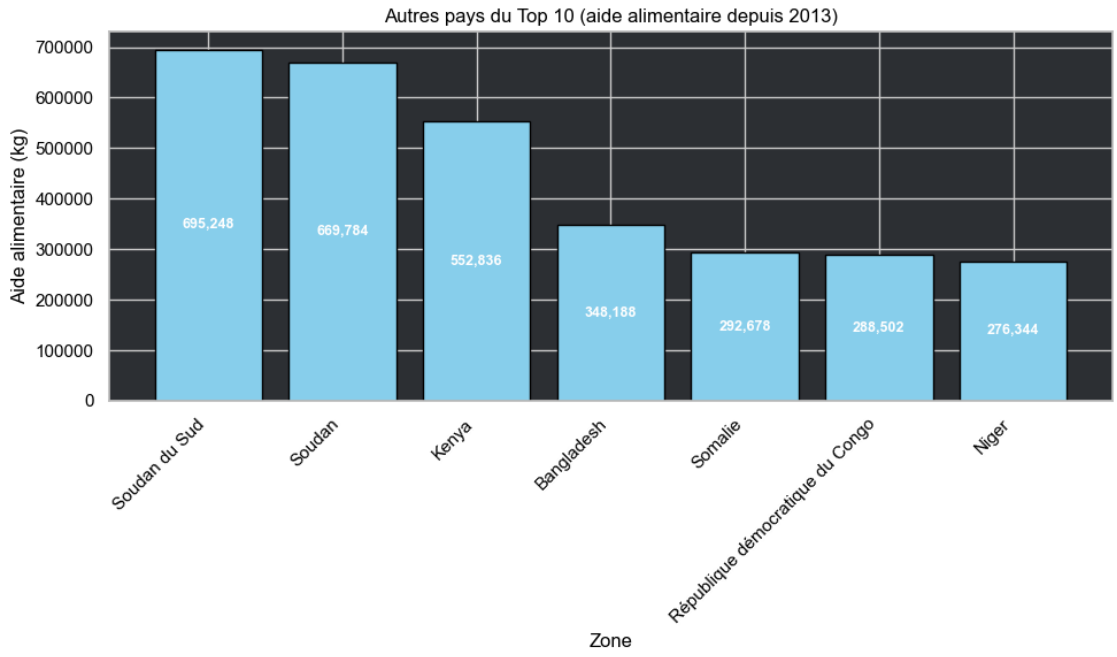
```
Top 10 des pays ayant reçu le plus d'aide alimentaire depuis 2013 :
Zone
République arabe syrienne           1858943
Éthiopie                            1381294
Yémen                               1206484
Soudan du Sud                        695248
Soudan                               669784
Kenya                                552836
Bangladesh                           348188
Somalie                              292678
République démocratique du Congo     288502
Niger                                276344
Name: Valeur, dtype: int64
```

[496]:
```
#Pour le PowerPoint
import pandas as pd
import matplotlib.pyplot as plt

# données DataFrame agrégé depuis 2013
data = {
    "Pays": [
        "République arabe syrienne", "Éthiopie", "Yémen", "Soudan du Sud",␣
↪"Soudan",
        "Kenya", "Bangladesh", "Somalie", "République démocratique du Congo",␣
↪"Niger"
    ],
    "Aide alimentaire (kg)": [
        1858943, 1381294, 1206484, 695248, 669784,
        552836, 348188, 292678, 288502, 276344
```

```
    ]
}
df = pd.DataFrame(data)
df_sorted = df.sort_values(by="Aide alimentaire (kg)", ascending=False)

# --- Création du podium pour les 3 premiers ---
top3 = df_sorted.head(3)

fig, ax = plt.subplots(figsize=(10, 6))

# Couleurs podium
colors = ["gold", "silver", "#cd7f32"]  # or, argent, bronze
widths = [0.6, 0.6, 0.6]
positions = [1, 0, 2]  # 1er au centre

# Hauteurs = quantités d'aide
heights = top3["Aide alimentaire (kg)"].values
max_height = max(heights)

# Ajustement de la hauteur du cadre pour éviter le chevauchement
ax.set_ylim(0, max_height * 1.25)

# Tracer les barres podium
for i, (pos, h, col) in enumerate(zip(positions, heights, colors)):
    ax.bar(pos, h, width=widths[i], color=col, edgecolor="black")

    # Ajouter les valeurs en blanc au milieu de la barre
    ax.text(
        pos, h/2, f"{top3['Pays'].iloc[i]}\n{h:,} kg",
        ha="center", va="center",
        fontsize=10, color="white", fontweight="bold"
    )

# Mise en forme
ax.set_xticks([])
ax.set_ylabel("Aide alimentaire cumulée (kg)")
ax.set_title("Podium des 3 pays les plus aidés depuis 2013")
ax.set_xlabel("Zone")
plt.tight_layout()
plt.show()

# --- Graphique complémentaire pour les 7 suivants ---

autres = df_sorted.iloc[3:]

plt.figure(figsize=(10,6))
bars = plt.bar(autres["Pays"], autres["Aide alimentaire (kg)"],
```

```
                color="skyblue", edgecolor="black")

plt.title("Autres pays du Top 10 (aide alimentaire depuis 2013)")
plt.ylabel("Aide alimentaire (kg)")
plt.xlabel("Zone")
plt.xticks(rotation=45, ha="right")

# Ajouter les valeurs en blanc sur les barres
for bar in bars:
    yval = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2, yval/2,   # position au milieu de la␣
 ↪barre
        f"{yval:,}",
        ha="center", va="center",
        fontsize=9, color="white", fontweight="bold"
    )

plt.tight_layout()
plt.show()
```





3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

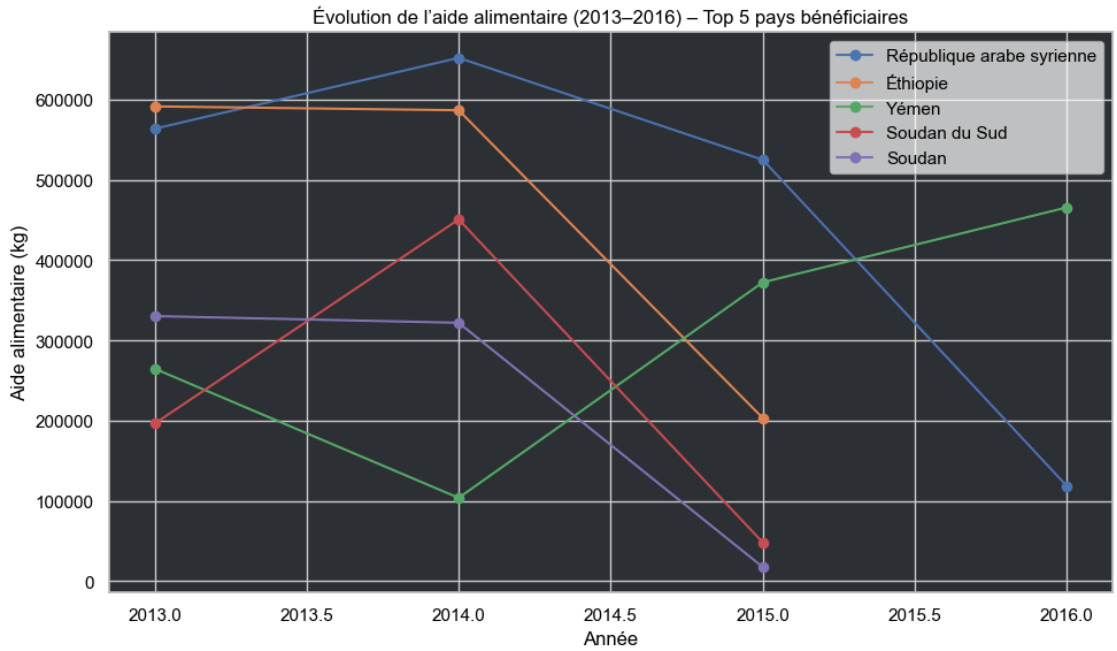
[497]:
```
#Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis␣
↪groupby sur zone et année
df = aide_alimentaire[["Zone", "Année", "Valeur"]]
aide_par_pays_annee = df.groupby(["Zone", "Année"])["Valeur"].sum().
↪reset_index()
```

[498]:
```
#Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de␣
↪l'aide alimentaire
aide_totale = aide_par_pays_annee.groupby("Zone")["Valeur"].sum()
top5_pays = aide_totale.sort_values(ascending=False).head(5).index.tolist()
```

[499]:
```
#On filtre sur le dataframe avec notre liste
df_top5 = aide_par_pays_annee[aide_par_pays_annee["Zone"].isin(top5_pays)]
```

[500]:
```
# Affichage des pays avec l'aide alimentaire par année
plt.figure(figsize=(10,6))
for pays in top5_pays:
    data_pays = df_top5[df_top5["Zone"] == pays]
    plt.plot(data_pays["Année"], data_pays["Valeur"], marker='o', label=pays)

plt.title("Évolution de l'aide alimentaire (2013-2016) - Top 5 pays␣
↪bénéficiaires")
plt.xlabel("Année")
plt.ylabel("Aide alimentaire (kg)")
```

```
# Légende avec fond blanc transparent
plt.legend(
    loc="upper right",
    frameon=True,
    facecolor="white",
    framealpha=0.7  # transparence du fond
)

plt.grid(True)
plt.tight_layout()
plt.show()
```



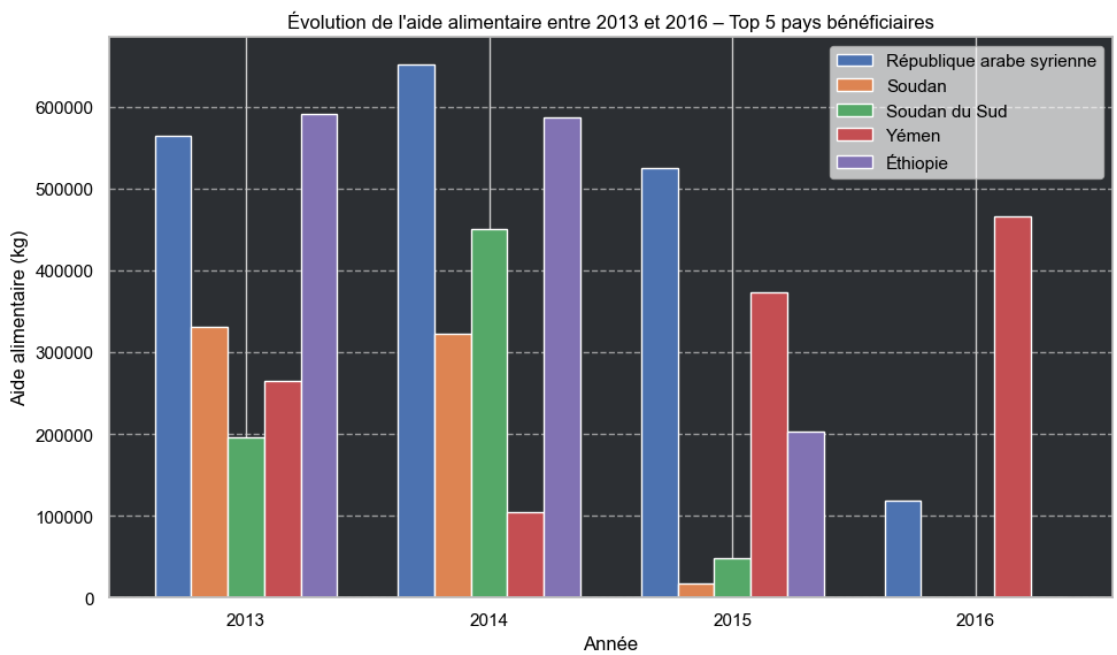
[501]:
```
import pandas as pd

# 1. Créer le DataFrame avec les colonnes utiles
df = aide_alimentaire[["Zone", "Année", "Valeur"]]

# 2. Agréger l'aide alimentaire par pays et année
aide_par_pays_annee = df.groupby(["Zone", "Année"])["Valeur"].sum().
 ↪reset_index()

# 3. Identifier les 5 pays les plus aidés sur la période 2013-2016
aide_totale = aide_par_pays_annee.groupby("Zone")["Valeur"].sum()
top5_pays = aide_totale.sort_values(ascending=False).head(5).index.tolist()
```

```
# 4. Filtrer le DataFrame sur ces 5 pays
df_top5 = aide_par_pays_annee[aide_par_pays_annee["Zone"].isin(top5_pays)]

# 5. Transformer en tableau avec les années en colonnes
tableau = df_top5.pivot(index="Zone", columns="Année", values="Valeur").
 ↪fillna(0)

# 6. Ajouter une colonne "Total" pour la période
tableau["Total 2013-2016"] = tableau.sum(axis=1)

# 7. Afficher le tableau
print("Évolution de l'aide alimentaire pour les 5 pays les plus bénéficiaires␣
 ↪(2013-2016) :")
print(tableau)
```

```
Évolution de l'aide alimentaire pour les 5 pays les plus bénéficiaires
(2013-2016) :
Année                          2013      2014      2015      2016  \
Zone
République arabe syrienne  563566.0  651870.0  524949.0  118558.0
Soudan                     330230.0  321904.0   17650.0       0.0
Soudan du Sud              196330.0  450610.0   48308.0       0.0
Yémen                      264764.0  103840.0  372306.0  465574.0
Éthiopie                   591404.0  586624.0  203266.0       0.0

Année                      Total 2013-2016
Zone
République arabe syrienne        1858943.0
Soudan                            669784.0
Soudan du Sud                     695248.0
Yémen                            1206484.0
Éthiopie                         1381294.0
```

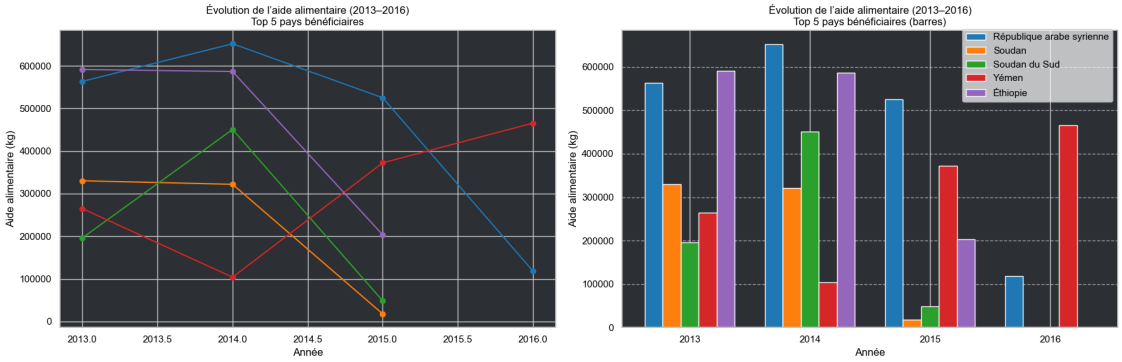
[503]:
```
import matplotlib.pyplot as plt
import numpy as np

# Données
annees = ['2013', '2014', '2015', '2016']
pays_donnees = {
    'République arabe syrienne': [563566, 651870, 524949, 118558],
    'Soudan': [330230, 321904, 17650, 0],
    'Soudan du Sud': [196330, 450610, 48308, 0],
    'Yémen': [264764, 103840, 372306, 465574],
    'Éthiopie': [591404, 586624, 203266, 0]
}
```

```python
# Paramètres du graphique
x = np.arange(len(annees))  # positions des années
width = 0.15  # largeur des barres

# Création du graphique
plt.figure(figsize=(10, 6))
for i, (pays, valeurs) in enumerate(pays_donnees.items()):
    plt.bar(x + i*width, valeurs, width, label=pays)

# Personnalisation
plt.xlabel('Année')
plt.ylabel("Aide alimentaire (kg)")
plt.title("Évolution de l'aide alimentaire entre 2013 et 2016 - Top 5 pays␣
 ↪bénéficiaires")
plt.xticks(x + width*2, annees)
# Légende avec fond blanc transparent
plt.legend(
    loc="upper right",
    frameon=True,
    facecolor="white",
    framealpha=0.7)  # transparence du fond
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
# Affichage
plt.show()
```



[504]:
```python
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# --- Définition des pays et des données ---
top5_pays = [
    "République arabe syrienne",
    "Soudan",
    "Soudan du Sud",
    "Yémen",
    "Éthiopie"
]

annees = ['2013', '2014', '2015', '2016']
pays_donnees = {
    'République arabe syrienne': [563566, 651870, 524949, 118558],
    'Soudan': [330230, 321904, 17650, 0],
    'Soudan du Sud': [196330, 450610, 48308, 0],
    'Yémen': [264764, 103840, 372306, 465574],
    'Éthiopie': [591404, 586624, 203266, 0]
}

# --- Palette Seaborn uniforme ---
palette = sns.color_palette("tab10", n_colors=len(top5_pays))
couleurs_pays = dict(zip(top5_pays, palette))
#sns.set_palette('pastel')
#sns.set_theme()

# --- Création des deux graphiques côte à côte ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18,6))

# 1) Graphique en courbes
for pays in top5_pays:
    data_pays = df_top5[df_top5["Zone"] == pays]
    ax1.plot(data_pays["Année"], data_pays["Valeur"],
             marker='o', label=pays, color=couleurs_pays[pays])

ax1.set_title("Évolution de l'aide alimentaire (2013-2016)\nTop 5 pays␣
 ↪bénéficiaires")
ax1.set_xlabel("Année")
ax1.set_ylabel("Aide alimentaire (kg)")
#ax1.legend()
ax1.grid(True)

# 2) Graphique en barres groupées
x = np.arange(len(annees))  # positions des années
width = 0.15  # largeur des barres
```

```
for i, (pays, valeurs) in enumerate(pays_donnees.items()):
    ax2.bar(x + i*width, valeurs, width,
            label=pays, color=couleurs_pays[pays])

ax2.set_xlabel("Année")
ax2.set_ylabel("Aide alimentaire (kg)")
ax2.set_title("Évolution de l'aide alimentaire (2013-2016)\nTop 5 pays␣
 ↪bénéficiaires (barres)")
ax2.set_xticks(x + width*2)
ax2.set_xticklabels(annees)
#ax2.legend()
ax2.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.legend(bbox_to_anchor=(1, 1.02),
    loc="upper right",
    frameon=True,
    facecolor="white",
    framealpha=0.7)  # transparence du fond
#plt.legend(bbox_to_anchor=(1, 1.02)) ##La même légende pour les deux graphiques
plt.show()
```



3.9 - Pays avec le moins de disponibilité par habitant

[366]:
```
print(dispo_alimentaire.columns)
```

```
Index(['Zone', 'Produit', 'Origine', 'Aliments pour animaux',
       'Autres Utilisations', 'Disponibilité alimentaire (Kcal/personne/jour)',
       'Disponibilité alimentaire en quantité (kg/personne/an)',
       'Disponibilité de matière grasse en quantité (g/personne/jour)',
       'Disponibilité de protéines en quantité (g/personne/jour)',
       'Disponibilité intérieure', 'Exportations - Quantité',
       'Importations - Quantité', 'Nourriture', 'Pertes', 'Production',
       'Semences', 'Traitement', 'Variation de stock'],
      dtype='object')
```

[505]:
```
####Calcul de la disponibilité en kcal par personne par jour par pays
# 1. Agréger la disponibilité alimentaire par pays
df_kcal = dispo_alimentaire.groupby("Zone")["Disponibilité alimentaire (Kcal/
↪personne/jour)"].sum().reset_index()

# 2. Trier les pays par disponibilité croissante
df_kcal_sorted = df_kcal.sort_values(by="Disponibilité alimentaire (Kcal/
↪personne/jour)").head(10)
```

[506]:
```
#Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
# Tri croissant pour les pays les moins bien dotés
df_kcal_sorted = df_kcal.sort_values(by="Disponibilité alimentaire (Kcal/
↪personne/jour)").head(10)

# Affichage
print("Top 10 des pays avec la plus faible disponibilité alimentaire par␣
↪habitant :")
print(df_kcal_sorted)
```

```
Top 10 des pays avec la plus faible disponibilité alimentaire par habitant :
                                          Zone  \
128                   République centrafricaine
166                                      Zambie
91                                  Madagascar
0                                  Afghanistan
65                                       Haïti
133  République populaire démocratique de Corée
151                                       Tchad
167                                    Zimbabwe
114                                     Ouganda
154                                  Timor-Leste

     Disponibilité alimentaire (Kcal/personne/jour)
128                                          1879.0
166                                          1924.0
91                                           2056.0
0                                            2087.0
65                                           2089.0
133                                          2093.0
151                                          2109.0
167                                          2113.0
114                                          2126.0
154                                          2129.0
```

[524]:
```
# Pour le PowerPoint
import matplotlib.pyplot as plt
```

```
# Données des 10 pays les moins dotés
pays = [
    "Centrafricaine", "Zambie", "Madagascar", "Afghanistan", "Haïti",
    "Corée du Nord", "Tchad", "Zimbabwe", "Ouganda", "Timor-Leste"
]
valeurs = [1879, 1924, 2056, 2087, 2089, 2093, 2109, 2113, 2126, 2129]

# Création du graphique
plt.figure(figsize=(18,6))
bars = plt.barh(pays, valeurs, color="green", edgecolor="black")

# Ajouter les valeurs en blanc au centre des barres
for bar, val in zip(bars, valeurs):
    plt.text(
        val/2, bar.get_y() + bar.get_height()/2,  # position au milieu de la␣
 ↪barre
        f"{val}", ha="center", va="center",
        color="white", fontsize=14, fontweight="bold"
    )

# Mise en forme
plt.xlabel("Disponibilité alimentaire (kcal/personne/jour)", fontweight="bold")
plt.ylabel("Zone")
plt.title("Top 10 des pays avec la plus faible disponibilité alimentaire par␣
 ↪habitant",fontsize=14, fontweight="bold")
plt.gca().invert_yaxis()  # pour que le pays le plus faible apparaisse en haut
plt.tight_layout()
plt.show()
```

3.10 - Pays avec le plus de disponibilité par habitant

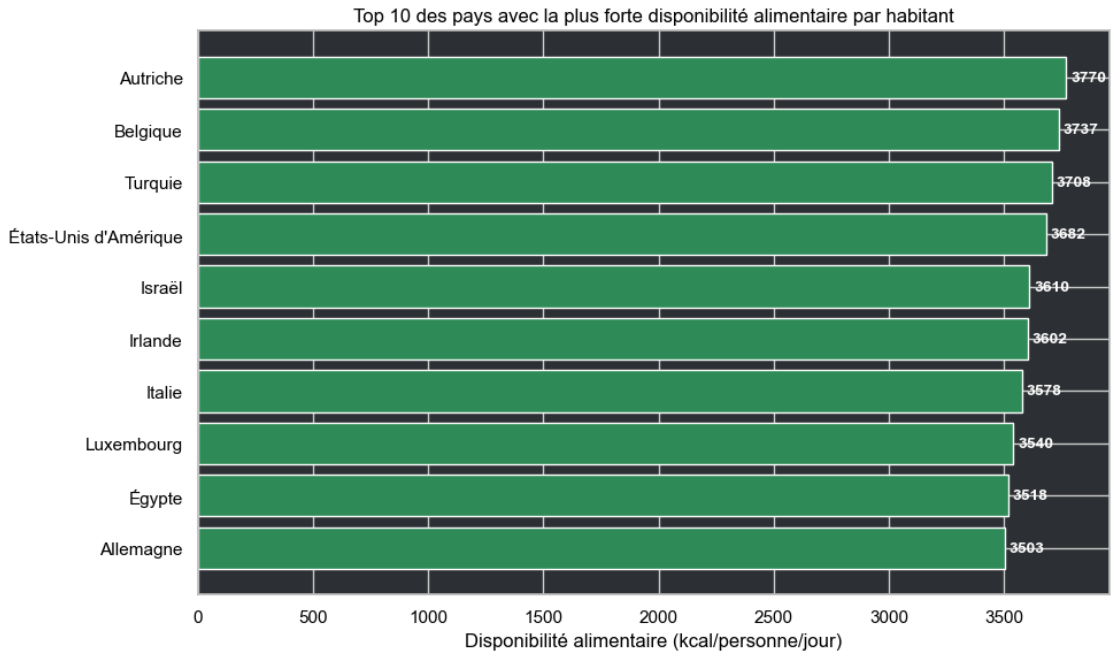
[370]:
```
#Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
import pandas as pd
```

```
# 1. Agréger la disponibilité alimentaire par pays
df_kcal = dispo_alimentaire.groupby("Zone")["Disponibilité alimentaire (Kcal/
↪personne/jour)"].sum().reset_index()

# 2. Trier les pays par disponibilité décroissante
df_kcal_sorted = df_kcal.sort_values(by="Disponibilité alimentaire (Kcal/
↪personne/jour)", ascending=False).head(10)

# 3. Affichage
print("Top 10 des pays avec la plus forte disponibilité alimentaire par␣
↪habitant :")
print(df_kcal_sorted)
```

```
Top 10 des pays avec la plus forte disponibilité alimentaire par habitant :
                      Zone  Disponibilité alimentaire (Kcal/personne/jour)
11                 Autriche                                          3770.0
16                 Belgique                                          3737.0
159                 Turquie                                          3708.0
171  États-Unis d'Amérique                                          3682.0
74                   Israël                                          3610.0
72                  Irlande                                          3602.0
75                   Italie                                          3578.0
89               Luxembourg                                          3540.0
168                  Égypte                                          3518.0
4                 Allemagne                                          3503.0
```

[528]:
```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Agréger la disponibilité alimentaire par pays
df_kcal = dispo_alimentaire.groupby("Zone")["Disponibilité alimentaire (Kcal/
↪personne/jour)"].sum().reset_index()

# 2. Trier les pays par disponibilité décroissante et garder les 10 premiers
df_top10 = df_kcal.sort_values(by="Disponibilité alimentaire (Kcal/personne/
↪jour)", ascending=False).head(10)

# 3. Création du graphique en barres horizontales
plt.figure(figsize=(10,6))
bars = plt.barh(df_top10["Zone"], df_top10["Disponibilité alimentaire (Kcal/
↪personne/jour)"], color="seagreen")

# 4. Ajouter les valeurs au bout des barres en blanc
for bar, val in zip(bars, df_top10["Disponibilité alimentaire (Kcal/personne/
↪jour)"]):
    plt.text(val + 20, bar.get_y() + bar.get_height()/2, f"{val:.0f}",
             va="center", color="white", fontsize=10, fontweight="bold")
```

```
# 5. Mise en forme
plt.xlabel("Disponibilité alimentaire (kcal/personne/jour)")
plt.title("Top 10 des pays avec la plus forte disponibilité alimentaire par␣
↪habitant")
plt.gca().invert_yaxis()  # pour que le pays le mieux doté apparaisse en haut
plt.tight_layout()
plt.show()
```



3.11 - Exemple de la Thaïlande pour le Manioc

[530]:
```
#création d'un dataframe avec uniquement la Thaïlande
import pandas as pd

# Filtrer uniquement la Thaïlande et le produit Manioc (et créer une COPIE)
df_thailande_manioc = dispo_alimentaire[
    (dispo_alimentaire["Zone"] == "Thaïlande") &
    (dispo_alimentaire["Produit"] == "Manioc")
].copy()

print("Données pour la Thaïlande (Manioc) :")
print(df_thailande_manioc[[
    "Zone", "Produit", "Production", "Exportations - Quantité",
    "Disponibilité intérieure", "Disponibilité alimentaire (Kcal/personne/jour)"
]])
```

```
Données pour la Thaïlande (Manioc) :
            Zone Produit    Production  Exportations - Quantité  \
13809  Thaïlande  Manioc  3.022800e+10             2.521400e+10

       Disponibilité intérieure  \
13809              6.264000e+09

       Disponibilité alimentaire (Kcal/personne/jour)
13809                                            40.0
```

[531]:
```
#Calcul de la sous nutrition en Thaïlande
prod_safe = df_thailande_manioc["Production"].replace(0, pd.NA)

# Calcul propre avec .loc et arrondi
df_thailande_manioc.loc[:, "Proportion exportée (%)"] = (
    df_thailande_manioc["Exportations - Quantité"] / prod_safe * 100
).round(2)

print("Exportations vs Production de Manioc en Thaïlande :")
print(df_thailande_manioc[[
    "Zone", "Produit", "Production", "Exportations - Quantité", "Proportion␣
 ↪exportée (%)"
]])
```

```
Exportations vs Production de Manioc en Thaïlande :
            Zone Produit    Production  Exportations - Quantité  \
13809  Thaïlande  Manioc  3.022800e+10             2.521400e+10

       Proportion exportée (%)
13809                    83.41
```

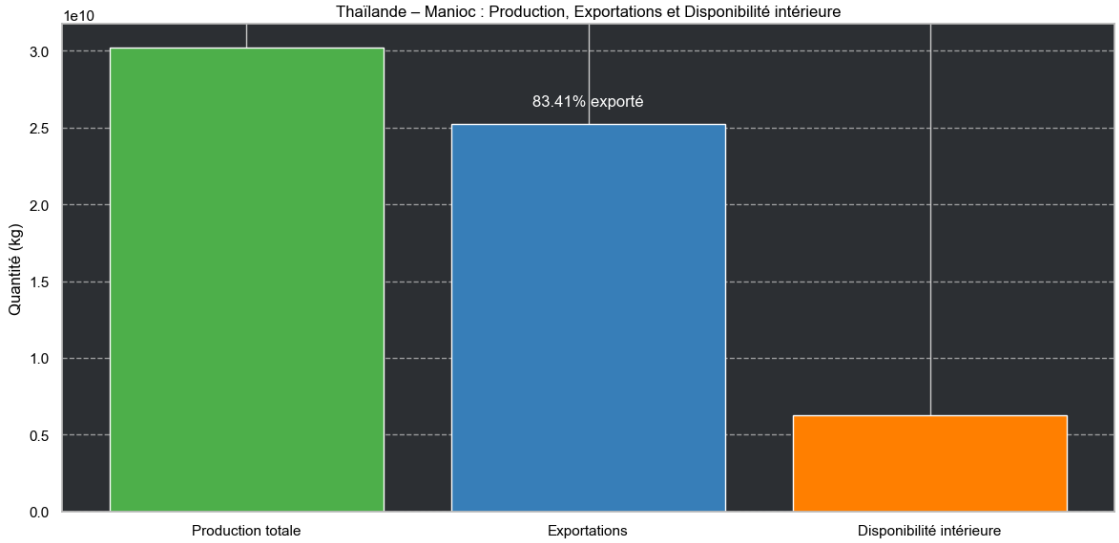
[532]:
```
# Part de personnes en état de sous-nutrition (dataset sous_nutrition)
df_thailande_sousnut = sous_nutrition[sous_nutrition["Zone"] == "Thaïlande"].
 ↪copy()

# Disponibilité alimentaire par habitant liée au manioc (si multi-lignes, on␣
 ↪peut sommer)
dispo_par_habitant_manioc = df_thailande_manioc["Disponibilité alimentaire␣
 ↪(Kcal/personne/jour)"].sum()

# Seuil nutritionnel de référence (indicatif)
seuil_nutritionnel = 2200

print("Sous-nutrition en Thaïlande :")
# Adapte les colonnes selon ton DF (exemple: 'Nombre de personnes en␣
 ↪sous-nutrition' ou '% sous-nutrition')
cols_existantes = [c for c in df_thailande_sousnut.columns if "sous" in c.
 ↪lower()]
```

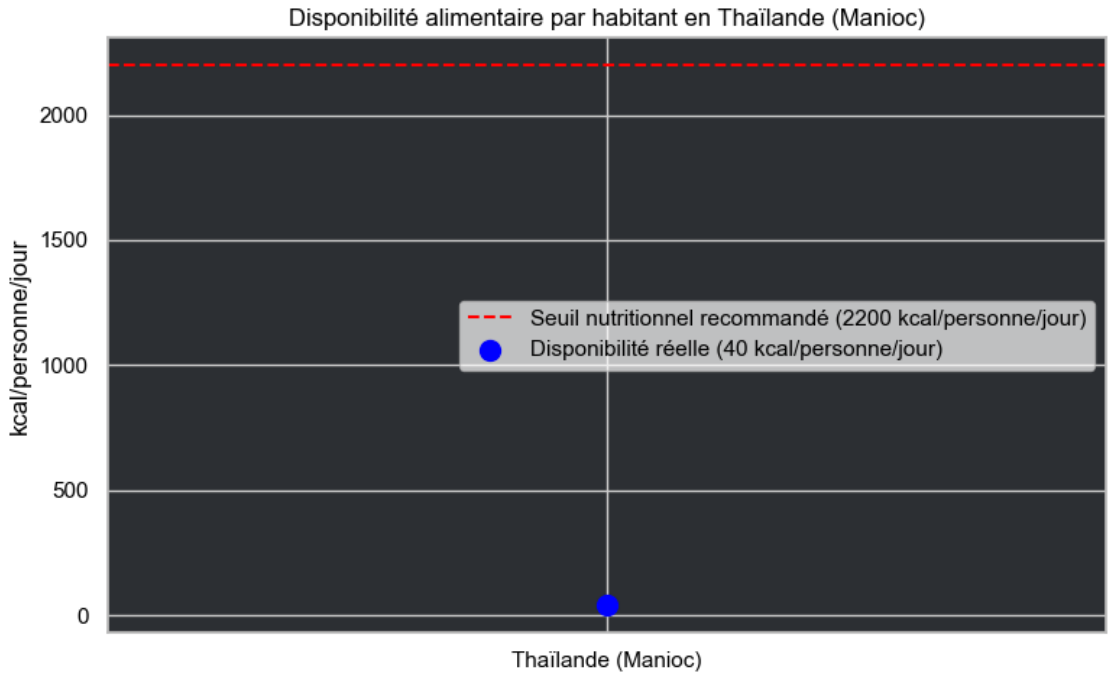
```
print(df_thailande_sousnut[["Zone"] + cols_existantes])

print(f"Disponibilité alimentaire par habitant en Thaïlande (Manioc) :␣
↪{dispo_par_habitant_manioc:.0f} kcal/personne/jour")
print(f"Comparaison au seuil {seuil_nutritionnel} kcal/jour : ",
      "en dessous" if dispo_par_habitant_manioc < seuil_nutritionnel else␣
↪"au-dessus")
```

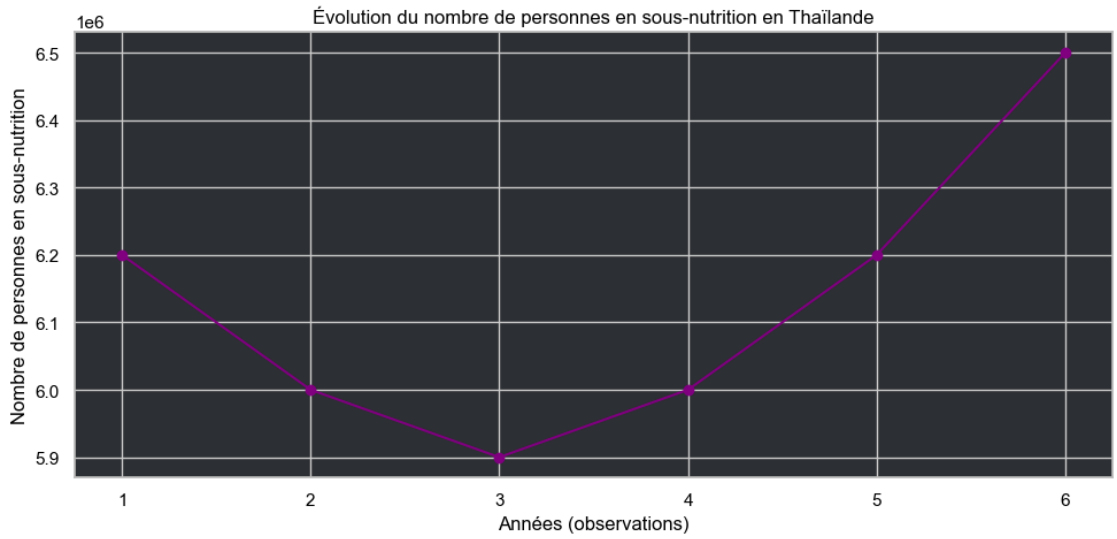
```
Sous-nutrition en Thaïlande :
           Zone  sous_nutrition
1110  Thaïlande       6200000.0
1111  Thaïlande       6000000.0
1112  Thaïlande       5900000.0
1113  Thaïlande       6000000.0
1114  Thaïlande       6200000.0
1115  Thaïlande       6500000.0
Disponibilité alimentaire par habitant en Thaïlande (Manioc) : 40
kcal/personne/jour
Comparaison au seuil 2200 kcal/jour :  en dessous
```

[538]:
```
#Pour le PowerPoint
import matplotlib.pyplot as plt

# Données principales
production = 3.0228e+10
exportations = 2.5214e+10
disponibilite_interieure = 6.264e+09
proportion_exportee = 83.41

# Données nutritionnelles
dispo_par_habitant = 40   # kcal/personne/jour
seuil_nutritionnel = 2200 # kcal/personne/jour
sous_nutrition = [6200000, 6000000, 5900000, 6000000, 6200000, 6500000]

# --- Graphique en barres pour production/exportations/disponibilité ---
plt.figure(figsize=(12,6))
categories = ["Production totale", "Exportations", "Disponibilité intérieure"]
valeurs = [production, exportations, disponibilite_interieure]
couleurs = ["#4daf4a", "#377eb8", "#ff7f00"]

bars = plt.bar(categories, valeurs, color=couleurs)

# Annotation de la proportion exportée
plt.text(1, exportations + 1e+9, f"{proportion_exportee:.2f}% exporté",
         ha="center", va="bottom", fontsize=12, color="white")

plt.ylabel("Quantité (kg)")
```

```
plt.title("Thaïlande - Manioc : Production, Exportations et Disponibilité␣
↪intérieure")
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.show()

# --- Graphique nutritionnel ---
plt.figure(figsize=(8,5))

# Ligne horizontale pour le seuil nutritionnel
plt.axhline(y=seuil_nutritionnel, color="red", linestyle="--",
            label="Seuil nutritionnel recommandé (2200 kcal/personne/jour)")

# Point pour la disponibilité réelle
plt.scatter(["Thaïlande (Manioc)"], [dispo_par_habitant], color="blue", s=100,␣
↪label="Disponibilité réelle (40 kcal/personne/jour)")

plt.title("Disponibilité alimentaire par habitant en Thaïlande (Manioc)")
plt.ylabel("kcal/personne/jour")
plt.legend(
    loc="center right",
    frameon=True,
    facecolor="white",
    framealpha=0.7)
plt.tight_layout()
plt.show()

# --- Graphique de la sous-nutrition ---
plt.figure(figsize=(10,5))
plt.plot(range(1, len(sous_nutrition)+1), sous_nutrition, marker="o",␣
↪color="purple")
plt.title("Évolution du nombre de personnes en sous-nutrition en Thaïlande")
plt.xlabel("Années (observations)")
plt.ylabel("Nombre de personnes en sous-nutrition")
plt.grid(True)
plt.tight_layout()
plt.show()
```

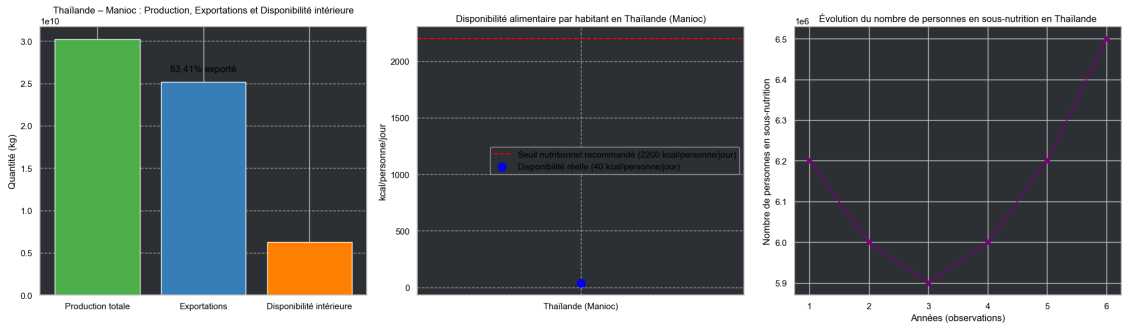

Thaïlande – Manioc : Production, Exportations et Disponibilité intérieure
1e10
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Quantité (kg)
83.41% exporté
Production totale
Exportations
Disponibilité intérieure


Disponibilité alimentaire par habitant en Thaïlande (Manioc)
2000
1500
1000
500
0
kcal/personne/jour
Seuil nutritionnel recommandé (2200 kcal/personne/jour)
Disponibilité réelle (40 kcal/personne/jour)
Thaïlande (Manioc)



[539]:
```
import matplotlib.pyplot as plt

# Données principales
production = 3.0228e+10
exportations = 2.5214e+10
disponibilite_interieure = 6.264e+09
proportion_exportee = 83.41

# Données nutritionnelles
dispo_par_habitant = 40   # kcal/personne/jour
seuil_nutritionnel = 2200 # kcal/personne/jour
sous_nutrition = [6200000, 6000000, 5900000, 6000000, 6200000, 6500000]

# Création des sous-graphiques côte à côte
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(20,6))

# --- Graphique en barres ---
categories = ["Production totale", "Exportations", "Disponibilité intérieure"]
valeurs = [production, exportations, disponibilite_interieure]
couleurs = ["#4daf4a", "#377eb8", "#ff7f00"]

bars = ax1.bar(categories, valeurs, color=couleurs)
ax1.text(1, exportations + 1e+9, f"{proportion_exportee:.2f}% exporté",
         ha="center", va="bottom", fontsize=12, color="black")
ax1.set_ylabel("Quantité (kg)")
ax1.set_title("Thaïlande - Manioc : Production, Exportations et Disponibilité␣
↪intérieure")
ax1.grid(axis="y", linestyle="--", alpha=0.7)
```

```
# --- Graphique nutritionnel ---
ax2.axhline(y=seuil_nutritionnel, color="red", linestyle="--",
            label="Seuil nutritionnel recommandé (2200 kcal/personne/jour)")
ax2.scatter(["Thaïlande (Manioc)"], [dispo_par_habitant],
            color="blue", s=100,
            label="Disponibilité réelle (40 kcal/personne/jour)")
ax2.set_title("Disponibilité alimentaire par habitant en Thaïlande (Manioc)")
ax2.set_ylabel("kcal/personne/jour")
ax2.legend()
ax2.grid(True, linestyle="--", alpha=0.7)

# --- Graphique sous-nutrition ---
ax3.plot(range(1, len(sous_nutrition)+1), sous_nutrition,
         marker="o", color="purple")
ax3.set_title("Évolution du nombre de personnes en sous-nutrition en Thaïlande")
ax3.set_xlabel("Années (observations)")
ax3.set_ylabel("Nombre de personnes en sous-nutrition")
ax3.grid(True)

plt.tight_layout()
plt.show()
```



Etape 6 - Analyse complémentaires

[377]:
```
#Ajouter en dessous toutes les analyses complémtaires
```

[540]:
```
pip install geodatasets
```

```
Requirement already satisfied: geodatasets in c:\users\ghysl\anaconda3\lib\site-
packages (2024.8.0)
Requirement already satisfied: pooch in c:\users\ghysl\anaconda3\lib\site-
packages (from geodatasets) (1.8.2)
Requirement already satisfied: platformdirs>=2.5.0 in
c:\users\ghysl\anaconda3\lib\site-packages (from pooch->geodatasets) (3.10.0)
Requirement already satisfied: packaging>=20.0 in
```

```
c:\users\ghysl\anaconda3\lib\site-packages (from pooch->geodatasets) (23.2)
Requirement already satisfied: requests>=2.19.0 in
c:\users\ghysl\anaconda3\lib\site-packages (from pooch->geodatasets) (2.32.2)
Requirement already satisfied: charset-normalizer<4,>=2 in
c:\users\ghysl\anaconda3\lib\site-packages (from
requests>=2.19.0->pooch->geodatasets) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in
c:\users\ghysl\anaconda3\lib\site-packages (from
requests>=2.19.0->pooch->geodatasets) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
c:\users\ghysl\anaconda3\lib\site-packages (from
requests>=2.19.0->pooch->geodatasets) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in
c:\users\ghysl\anaconda3\lib\site-packages (from
requests>=2.19.0->pooch->geodatasets) (2025.10.5)
Note: you may need to restart the kernel to use updated packages.
```

[541]:
```python
#Charger les données et la carte du monde
import geopandas as gpd
import geodatasets
import pandas as pd
import seaborn as snb

# Charger les pays (Natural Earth 1:110m)
world = gpd.read_file(geodatasets.get_path("naturalearth.land"))
```

[547]:
```python
# Import libraries
import geopandas as gpd

# Use the direct URL to the Natural Earth data instead of the deprecated␣
↪datasets module
# This downloads the Natural Earth low resolution cultural vectors directly
world = gpd.read_file(
    "https://naciscdn.org/naturalearth/110m/cultural/ne_110m_admin_0_countries.
↪zip"
)

print(world.columns)  # Verify available columns
print(world.head())
```

```
Index(['featurecla', 'scalerank', 'LABELRANK', 'SOVEREIGNT', 'SOV_A3',
       'ADM0_DIF', 'LEVEL', 'TYPE', 'TLC', 'ADMIN',
       …
       'FCLASS_TR', 'FCLASS_ID', 'FCLASS_PL', 'FCLASS_GR', 'FCLASS_IT',
       'FCLASS_NL', 'FCLASS_SE', 'FCLASS_BD', 'FCLASS_UA', 'geometry'],
      dtype='object', length=169)
        featurecla  scalerank  LABELRANK                   SOVEREIGNT SOV_A3  \
0  Admin-0 country          1          6                         Fiji    FJI
```

```
1  Admin-0 country          1          3  United Republic of Tanzania    TZA
2  Admin-0 country          1          7               Western Sahara    SAH
3  Admin-0 country          1          2                       Canada    CAN
4  Admin-0 country          1          2     United States of America    US1

   ADM0_DIF  LEVEL               TYPE TLC                        ADMIN  …  \
0         0      2  Sovereign country   1                         Fiji  …
1         0      2  Sovereign country   1  United Republic of Tanzania  …
2         0      2      Indeterminate   1               Western Sahara  …
3         0      2  Sovereign country   1                       Canada  …
4         1      2            Country   1     United States of America  …

      FCLASS_TR     FCLASS_ID     FCLASS_PL FCLASS_GR  FCLASS_IT  \
0          None          None          None      None       None
1          None          None          None      None       None
2  Unrecognized  Unrecognized  Unrecognized      None       None
3          None          None          None      None       None
4          None          None          None      None       None

      FCLASS_NL FCLASS_SE  FCLASS_BD FCLASS_UA  \
0          None      None       None      None
1          None      None       None      None
2  Unrecognized      None       None      None
3          None      None       None      None
4          None      None       None      None

                                            geometry
0  MULTIPOLYGON (((180 -16.067, 180 -16.555, 179…
1  POLYGON ((33.904 -0.95, 34.073 -1.0598, 37.699…
2  POLYGON ((-8.6656 27.656, -8.6651 27.589, -8.6…
3  MULTIPOLYGON (((-122.84 49, -122.97 49.003, -1…
4  MULTIPOLYGON (((-122.84 49, -120 49, -117.03 4…

[5 rows x 169 columns]
```

[548]:
```python
# Harmoniser les noms si nécessaire
mapping = {
    "République centrafricaine": "Central African Republic",
    "République démocratique du Congo": "Democratic Republic of the Congo",
    "République arabe syrienne": "Syria",
    "États-Unis d'Amérique": "United States of America",
    "Côte d'Ivoire": "Ivory Coast"
}
df_2017["Zone"] = df_2017["Zone"].replace(mapping)
print(world.columns)

# Fusionner la carte avec tes données
```

```
world_data = world.merge(df_2017, left_on="NAME", right_on="Zone", how="left")
```

```
Index(['featurecla', 'scalerank', 'LABELRANK', 'SOVEREIGNT', 'SOV_A3',
       'ADM0_DIF', 'LEVEL', 'TYPE', 'TLC', 'ADMIN',
       …
       'FCLASS_TR', 'FCLASS_ID', 'FCLASS_PL', 'FCLASS_GR', 'FCLASS_IT',
       'FCLASS_NL', 'FCLASS_SE', 'FCLASS_BD', 'FCLASS_UA', 'geometry'],
      dtype='object', length=169)
```

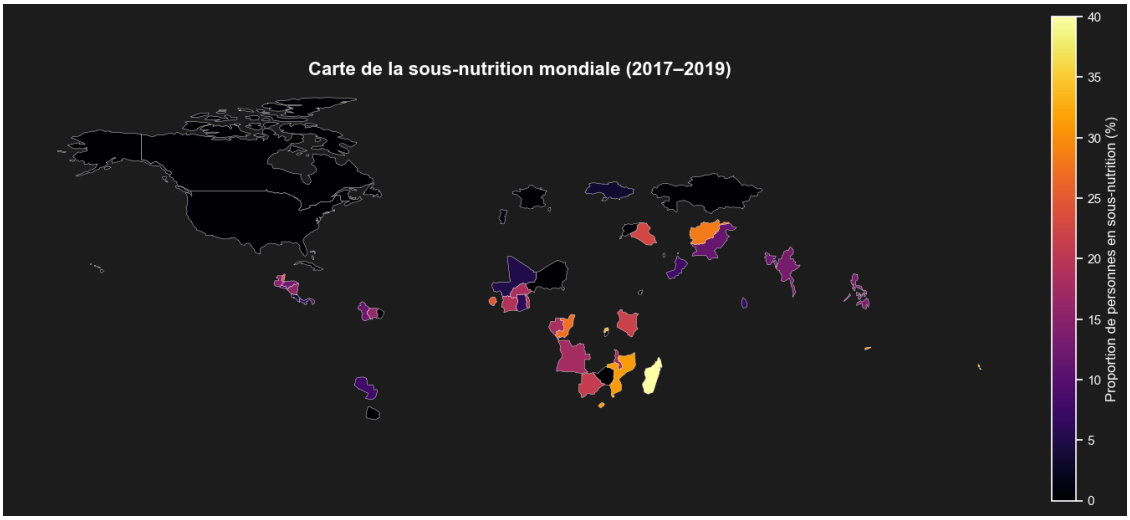
[594]:
```
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots(1, 1, figsize=(15, 10))
world_data.plot(column="proportion_%", legend=True, ax=ax,
                legend_kwds={"label": "Proportion de personnes en␣
 ↪sous-nutrition (%)"})

plt.title("Sous-nutrition mondiale (2017-2019)")
#sns.set_palette('dark')
sns.set_theme(style='whitegrid', palette='pastel')
#sns.set_theme()
plt.axis("off")
plt.show()
```

[614]:
```
 ###La Crte précédeente est bonne. mais les fnd blance est genant.
###Je choisis de rendre ma carte plus lisible et présentable.
```

[615]:
```
#changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={'Pays bénéficiaire': 'Zone'}, inplace=True)
```

[617]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib as mpl

# Thème sombre
sns.set_theme(style='darkgrid')

# Création du graphique
fig, ax = plt.subplots(figsize=(15, 10))
fig.patch.set_facecolor('#1c1c1c')
ax.set_facecolor('#1c1c1c')

# Palette et normalisation
cmap = plt.cm.inferno
norm = mpl.colors.Normalize(vmin=0, vmax=40)

# Tracer la carte sans légende automatique
world_data.plot(
    column="proportion_%",
    cmap=cmap,
    ax=ax,
    edgecolor="white",
    linewidth=0.3
)

# Création manuelle de la colorbar
sm = mpl.cm.ScalarMappable(cmap=cmap, norm=norm)
sm._A = []  # nécessaire pour matplotlib < 3.1
cbar = fig.colorbar(sm, ax=ax, orientation="vertical", shrink=0.6, pad=0.02)

# Personnalisation de la colorbar
cbar.set_label("Proportion de personnes en sous-nutrition (%)", color="white",␣
 ↪fontsize=12)
cbar.ax.yaxis.set_tick_params(color="white")
plt.setp(plt.getp(cbar.ax.axes, 'yticklabels'), color='white')

# Titre en blanc
plt.title("Carte de la sous-nutrition mondiale (2017-2019)", fontsize=16,␣
 ↪fontweight="bold", color="white")
```

```
plt.axis("off")
plt.tight_layout()
plt.show()
```



[ ]:

```
```